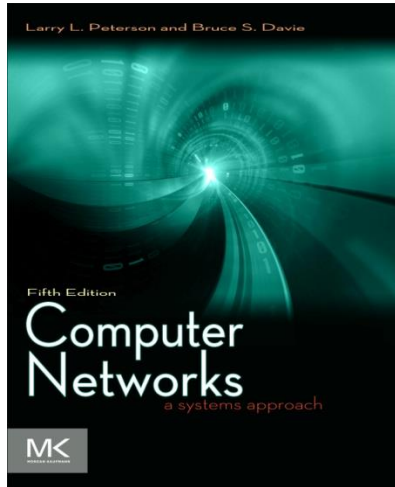




Chapter 6

Congestion Control and Resource Allocation

Problem

- Problem- Allocating Resources
 - How to *effectively* and *fairly* allocate resources among a collection of competing users?
 - Spans across the entire protocol stack

Chapter Outline

- Issues in Resource Allocation
- Queuing Disciplines
- TCP Congestion Control
- Congestion Avoidance Mechanism
- Quality of Service

Congestion Control and Resource Allocation

- Resources
 - Bandwidth of the links
 - Buffers at the routers and switches
- Packets contend at a router for the use of a link, with each contending packet placed in a queue waiting for its turn to be transmitted over the link

Congestion Control and Resource Allocation

- When too many packets are contending for the same link
 - The queue overflows
 - Packets experience higher end-to-end delay
 - Packets get dropped
 - **Network is congested!**- when long queues persist and drops become common

Congestion Control and Resource Allocation

- If the network takes active role in allocating resources
 - The congestion may be avoided
 - No need for congestion control
- But, allocating resources with precision is difficult
 - Resources are distributed throughout the network

Congestion Control and Resource Allocation

- On the other hand, let the sources send as much data as they want, then recover from the congestion when it occurs
- Easier approach but it can be disruptive as many packets may be discarded before congestion is controlled

Congestion Control and Resource Allocation

- In network elements
 - Various queuing disciplines used to control the order in which packets get transmitted
- At the hosts' end
 - The congestion control mechanism paces how fast sources are allowed to send packets

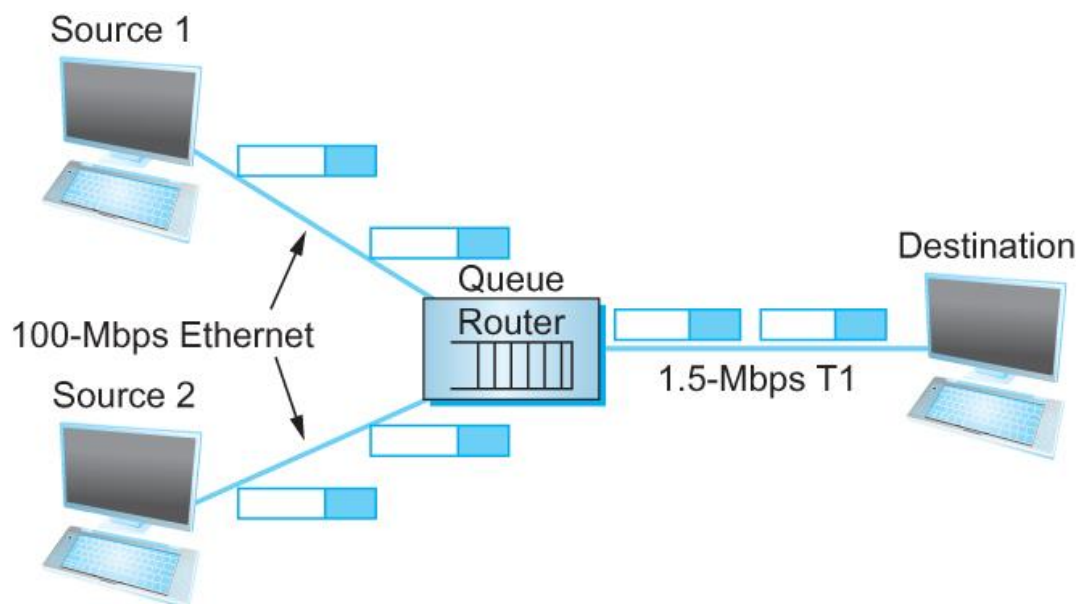
ISSUES IN RESOURCE ALLOCATION

Issues in Resource Allocation

- Network Model
 - Packet Switched Network
 - In packet-switched network (or internet) consisting of multiple links and switches (or routers),
 - A given source may have more than enough capacity on the immediate outgoing link to send a packet, but somewhere in the network, its packets encounter a link that is being used by many different traffic sources

Issues in Resource Allocation

- Network Model
 - Packet Switched Network



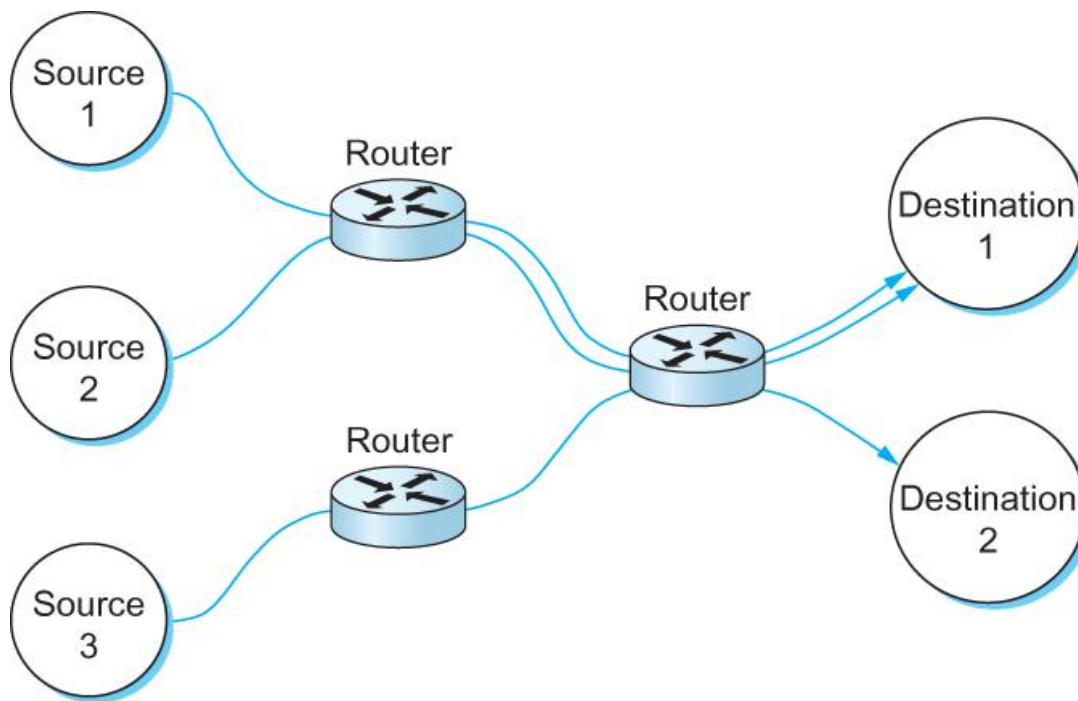
A potential bottleneck router.

Issues in Resource Allocation

- Network Model
 - Connectionless Flows
 - Assume that the network is connectionless, with any connection-oriented service implemented in the transport protocol that is running on the end hosts.
 - The datagrams are switched independently, but it is usually the case that a stream of datagrams between a particular pair of hosts flows through a particular set of routers

Issues in Resource Allocation

- Network Model
 - Connectionless Flows



Multiple flows passing through a set of routers

Issues in Resource Allocation

- Network Model

- Connectionless Flows

- One of the powers of the flow abstraction is that flows can be defined at different granularities.
 - For example, a flow can be host-to-host or process-to-process.
 - In the latter case, a flow is essentially the same as a channel.
 - The difference being a flow is visible to the routers inside the network, whereas a channel is an end-to-end abstraction.

Issues in Resource Allocation

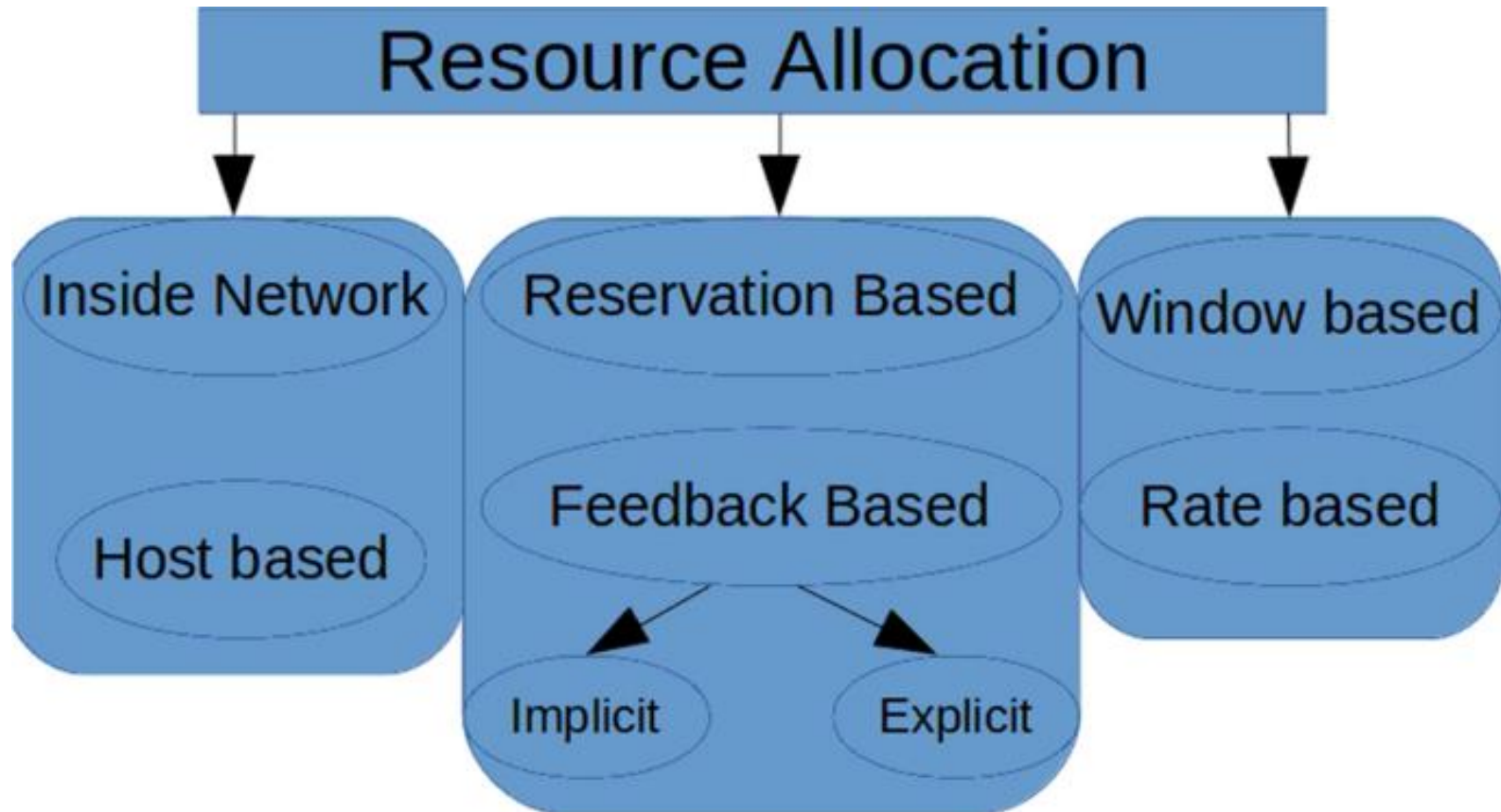
- Network Model
 - Connectionless Flows
 - Because multiple related packets flow through each router, it sometimes makes sense to maintain some state information for each flow. This state is sometimes called soft state.
 - The main difference between soft state and “hard” state is that soft state need not always be explicitly created and removed by signaling.

Issues in Resource Allocation

- Network Model
 - Connectionless Flows
 - Soft state represents a middle ground between a purely connectionless network and a purely connection-oriented network.
 - In general, when a packet happens to belong to a flow for which the router is currently maintaining soft state, then the router is better able to handle the packet.

Issues in Resource Allocation

- Taxonomy



Issues in Resource Allocation

- Taxonomy
 - Router-centric versus Host-centric
 - In a router-centric design, each router decides when packets are forwarded and selects which packets are forwarded and which are dropped, as well as for informs the hosts of they are allowed to send.
 - In a host-centric design, the end hosts observe the network conditions (e.g., how many packets they are successfully getting through the network) and adjust their behavior accordingly.

Issues in Resource Allocation

- Taxonomy

- Reservation-based versus Feedback-based

- In a reservation-based system, some entity asks the network for a certain amount of capacity to be allocated for a flow.
 - In a feedback-based approach, the end hosts begin sending data without first reserving any capacity and then adjust their sending rate according to the feedback they receive.
 - This feedback can either be *explicit* (i.e., a congested router sends a “please slow down” message to the host) or it can be *implicit* (i.e., the end host adjusts its sending rate according to the externally observable behavior of the network, such as packet losses).

Issues in Resource Allocation

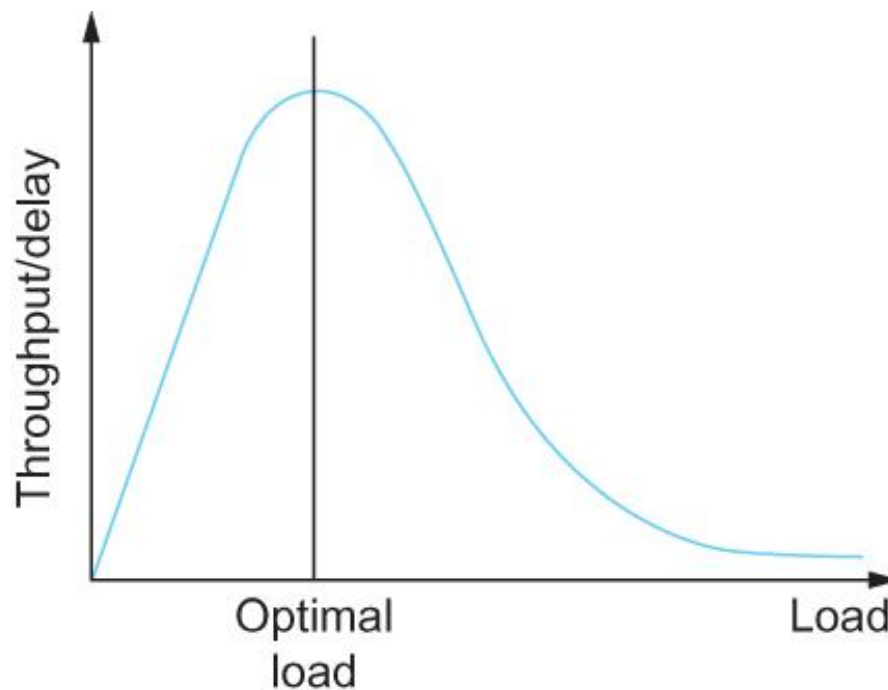
- Taxonomy
 - Window-based versus Rate-based
 - Window advertisement is used within the network to reserve buffer space.
 - Control sender's behavior using a rate, how many bit per second the receiver or network is able to absorb.

Issues in Resource Allocation

- Evaluation Criteria
 - Effective Resource Allocation
 - Two principal metrics of networking: maximize throughput and minimize delay.
 - One sure way to increase throughput is to allow as many packets into the network as possible, driving the utilization of all the links up to 100%.
 - The problem, increasing the number of packets also increases the length of the queues at each router. Longer queues, in turn, mean packets are delayed longer in the network

Issues in Resource Allocation

- Evaluation Criteria
 - Effective Resource Allocation



Ratio of throughput to delay as a function of load

Issues in Resource Allocation

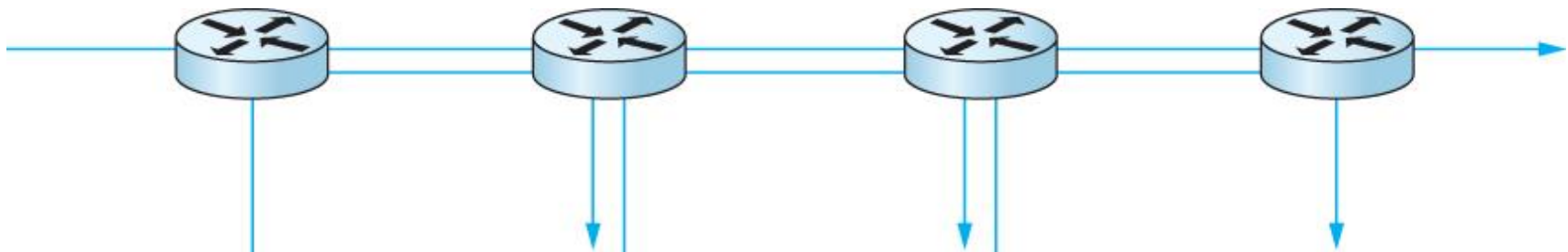
- Evaluation Criteria
 - Fair Resource Allocation
 - For example, a reservation-based resource allocation scheme provides an explicit way to create controlled unfairness.
 - With such a scheme, one might use reservations to enable a video stream to receive 1 Mbps across some link while a file transfer receives only 10 Kbps over the same link.

Issues in Resource Allocation

- Evaluation Criteria
 - Fair Resource Allocation
 - When several flows share a particular link, each flow should receive an equal share of the bandwidth.
 - This definition presumes that a *fair share of bandwidth means an equal share of bandwidth*.
 - But even in the absence of reservations, equal shares may not equate to fair shares.

Issues in Resource Allocation

- Evaluation Criteria
 - Fair Resource Allocation



One four-hop flow competing with three one-hop flows

Issues in Resource Allocation

- Evaluation Criteria

- Fair Resource Allocation

- Given a set of flow throughputs ($x_1, x_2, \dots, x_n$) (measured inconsistent units such as bits/second),
 - Assuming that fair implies equal and that all paths are of equal length, the following function assigns a fairness index to the flows:

$$f(x_1, x_2, \dots, x_n) = \frac{(\sum_{i=1}^n x_i)^2}{n \sum_{i=1}^n x_i^2}$$

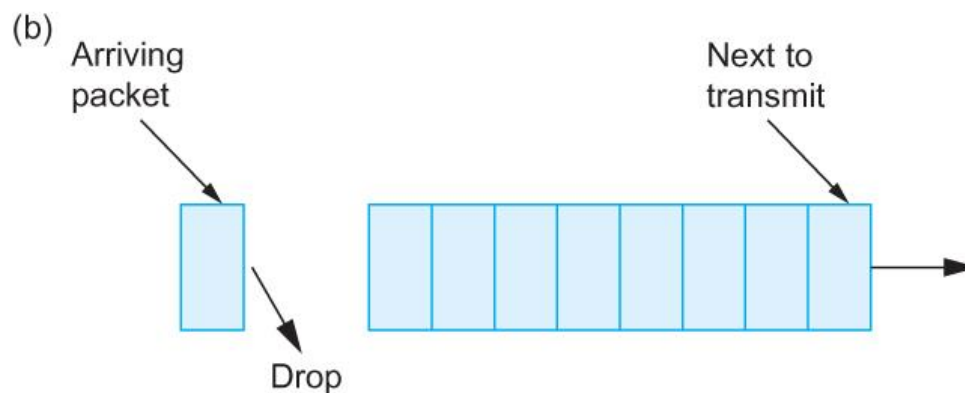
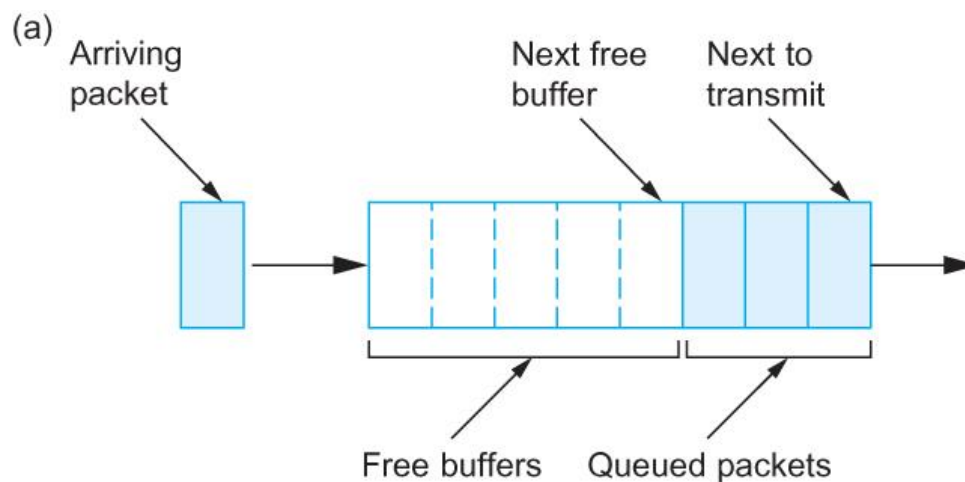
- The fairness index always results in a number between 0 and 1, with 1 representing greatest fairness.

QUEUEING DISCIPLINES

Queuing Disciplines

- FIFO queuing / FCFS queuing:
 - The first packet that arrives at a router is the first packet to be transmitted.
 - If a packet arrives and the queue (buffer space) is full, then the router discards that packet.
 - This is done without regard to which flow the packet belongs to or how important the packet is. This is called *tail drop*.

Queuing Disciplines



(a) FIFO queuing; (b) tail drop at a FIFO queue.

Queuing Disciplines

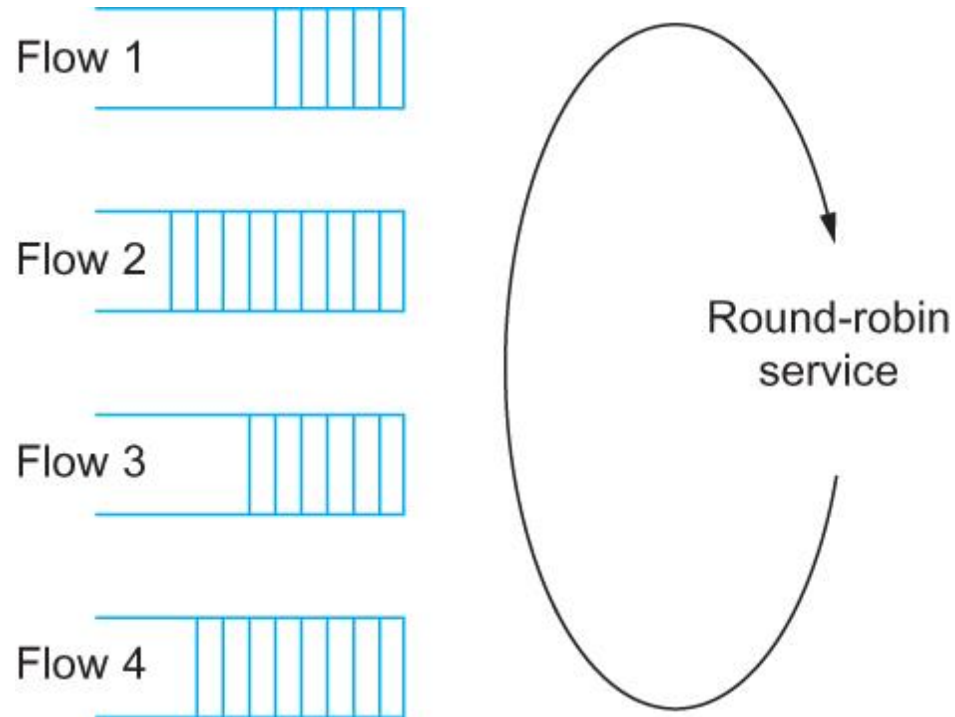
- A variation on basic FIFO queuing is priority queuing by marking each packet with a priority.
- The routers then implement multiple FIFO queues, one for each priority class.
- The router always transmits packets out of the highest-priority queue if that queue is non-empty before moving on to the next priority queue.
- Within each priority, packets are still managed in a FIFO manner.

Queuing Disciplines

- Fair Queuing
 - The main problem with FIFO queuing is that it does not separate packets according to the flow to which they belong.
 - Fair queuing (FQ) is an algorithm that maintains a separate queue for each flow currently being handled by the router.
 - The router then services these queues in a sort of round-robin,

Queuing Disciplines

■ Fair Queuing



Round-robin service of four flows at a router

Queuing Disciplines

- Fair Queuing
 - The main complication with Fair Queuing is that the packets being processed at a router are not necessarily the same length.
 - To truly allocate the bandwidth of the outgoing link in a fair manner, it is necessary to take packet length into consideration.

Queuing Disciplines

- Fair Queuing

- For example, if a router is managing two flows, one with 1000-byte packets and the other with 500-byte packets (perhaps because of fragmentation upstream from this router), then a simple round-robin servicing of packets from each flow's queue will give the first flow two thirds of the link's bandwidth and the second flow only one-third of its bandwidth.

Queuing Disciplines

- Fair Queuing
 - What's needed is bit-by-bit round-robin; that is, the transmit a bit from flow 1, then a bit from flow 2, and so on.
 - The FQ mechanism simulates this by determining when a given packet would finish being transmitted if it were being sent using bit-by-bit round-robin, and then uses this finishing time to sequence the packets for transmission.

Queuing Disciplines

- Fair Queuing
 - For this flow, let
 - P_i : denote the length of packet i
 - S_i : time when the router starts to transmit packet i
 - F_i : time when router finishes transmitting packet i
 - Clearly, $F_i = S_i + P_i$

Queuing Disciplines

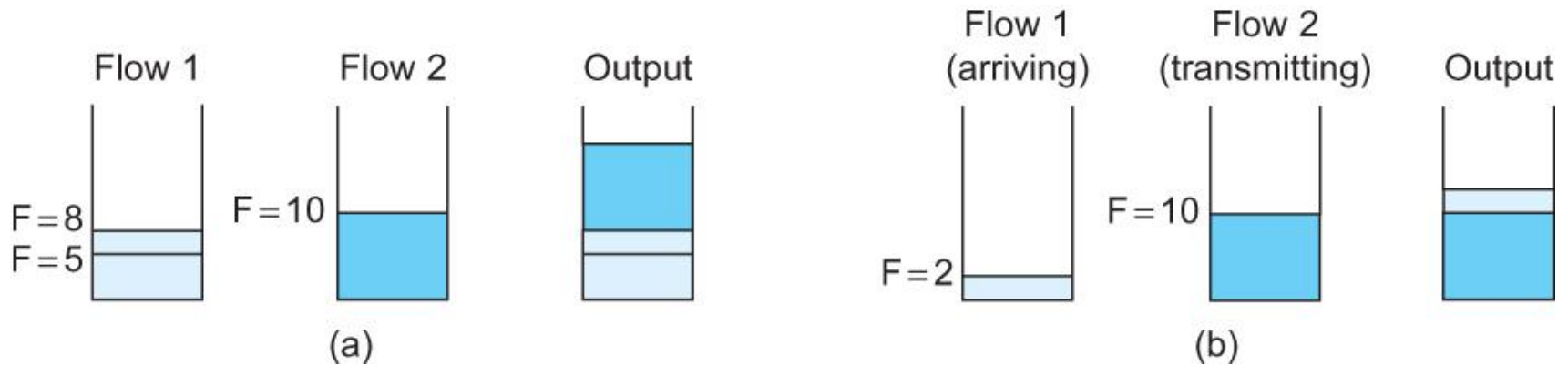
- Fair Queuing
 - When do we start transmitting packet i ?
 - Depends on whether packet i arrived before or after the router finishes transmitting packet $i-1$ for the flow
 - Let A_i denote the time that packet i arrives at the router
 - Then $S_i = \max(F_{i-1}, A_i)$
 - $F_i = \max(F_{i-1}, A_i) + P_i$

Queuing Disciplines

- Fair Queuing
 - Next packet to transmit is always the packet that has the lowest F_i
 - The packet that should finish transmission before all others

Queuing Disciplines

■ Fair Queuing



Example of fair queuing in action: (a) packets with earlier finishing times are sent first; (b) sending of a packet already in progress is completed

TCP CONGESTION CONTROL

TCP Congestion Control

- TCP congestion control was introduced into the Internet in the late 1980s by Van Jacobson, roughly eight years after the TCP/IP protocol stack had become operational.
- Immediately preceding this time, the Internet was suffering from congestion collapse

TCP Congestion Control

- The idea of TCP congestion control is for each source to determine how much capacity is available in the network, so that it knows how many packets it can safely have in transit.

TCP Congestion Control

- Once a source has some packets in transit, it uses the arrival of an ACK as a signal that one of its packets has left the network → **safe** to insert a new packet into the network without congestion
- By using ACKs to pace the transmission of packets, TCP is said to be *self-clocking*.

TCP Congestion Control

- Additive Increase Multiplicative Decrease
 - TCP maintains a new variable, called *CongestionWindow*, which is used to limit how much data it is allowed in transit at a given time.
 - TCP is modified such that the maximum number of bytes of unacknowledged data allowed is now the minimum of the congestion window and the advertised window.

TCP Congestion Control

- Additive Increase Multiplicative Decrease
 - How does the source determine that the network is congested and that it should decrease the congestion window?
 - It is rare that a packet is dropped because of an error during transmission.
 - Therefore, TCP interprets timeouts as a sign of congestion and reduces the rate at which it is transmitting.

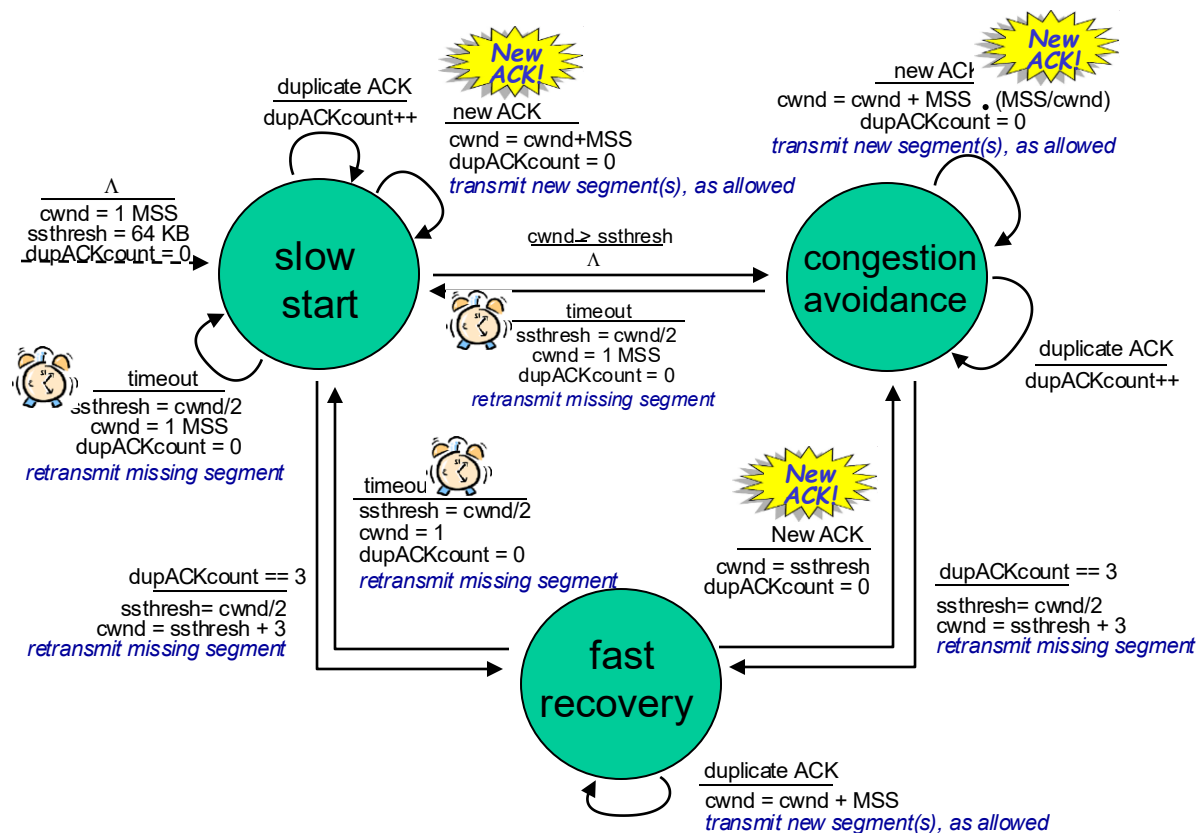
TCP Congestion Control

- Additive Increase Multiplicative Decrease
 - Each time a timeout occurs, the source sets CongestionWindow to half of its previous value.
 - This halving of the CongestionWindow for each timeout corresponds to the “multiplicative decrease” part of AIMD.

TCP Congestion Control

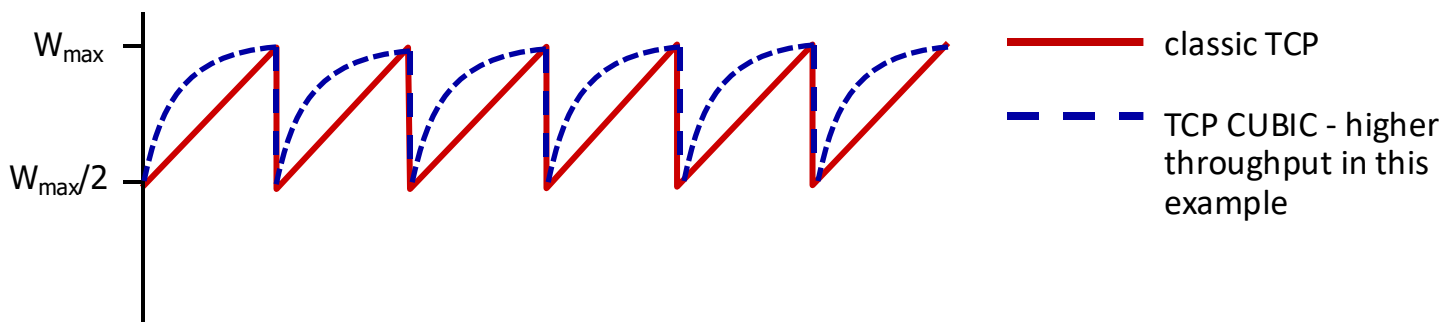
- Additive Increase Multiplicative Decrease
 - We also need to be able to increase the congestion window to take advantage of newly available capacity in the network.
 - This is the job of “additive increase” part of AIMD.

TCP congestion control



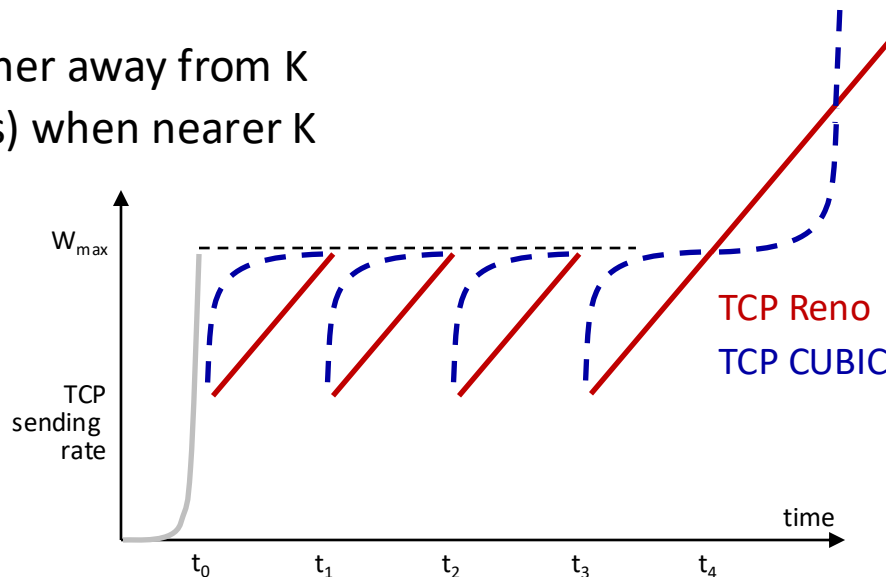
TCP CUBIC

- Is there a better way than AIMD to “probe” for usable bandwidth?
- Insight/intuition:
 - $W_{\max}$: sending rate at which congestion loss was detected
 - congestion state of bottleneck link probably (?) hasn't changed much
 - after cutting rate/window in half on loss, initially ramp to $W_{\max}$ *faster*, but then approach $W_{\max}$ more *slowly*



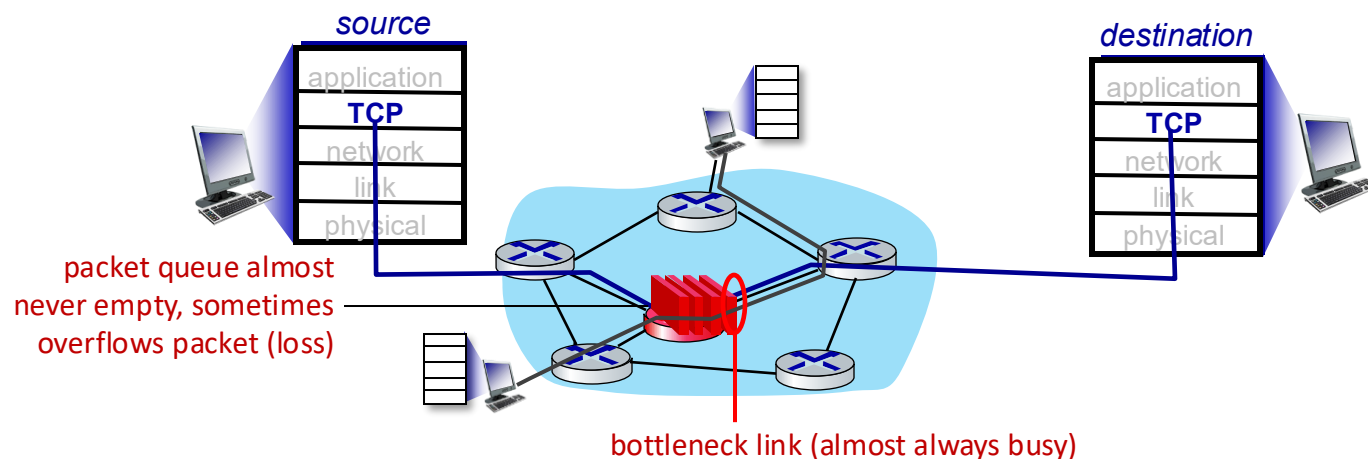
TCP CUBIC

- K: point in time when TCP window size will reach $W_{\max}$
 - K itself is tunable
- increase W as a function of the *cube* of the distance between current time and K
 - larger increases when further away from K
 - smaller increases (cautious) when nearer K
- TCP CUBIC default in Linux, most popular TCP for popular Web servers (until ~2024)



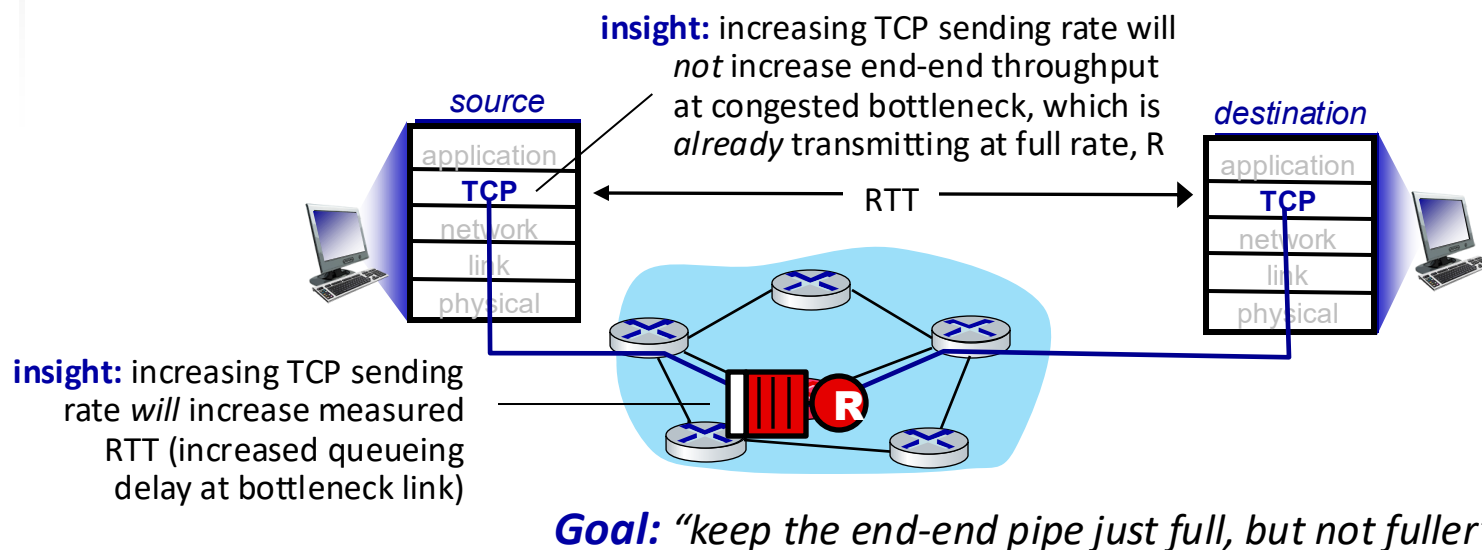
TCP and the congested “bottleneck link”

- TCP (classic, CUBIC) increase TCP’s sending rate until packet loss occurs at some router’s output: the *bottleneck link*



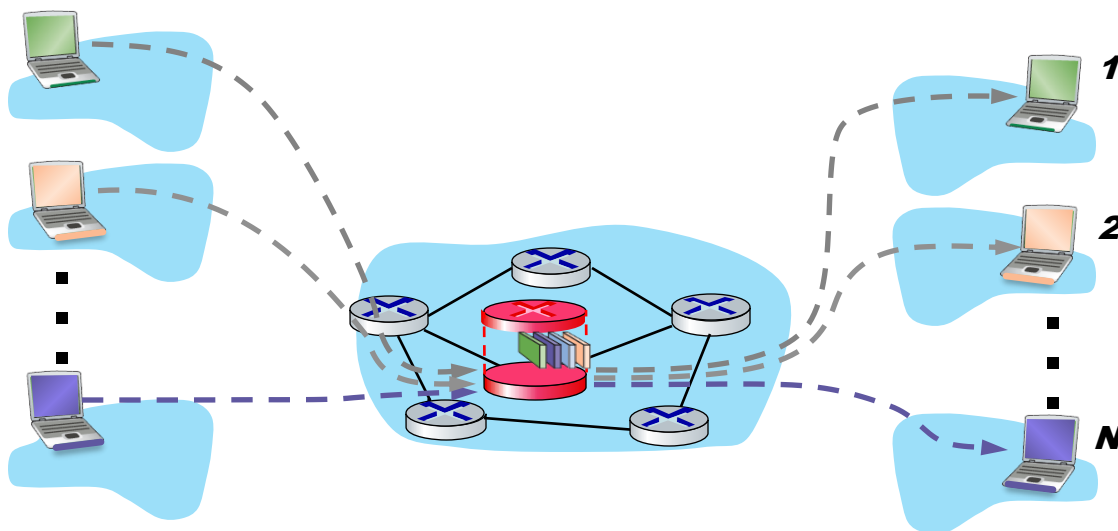
TCP and the congested “bottleneck link”

- TCP (classic, CUBIC) increase TCP’s sending rate until packet loss occurs at some router’s output: the *bottleneck link*
- understanding congestion: useful to focus on congested bottleneck link



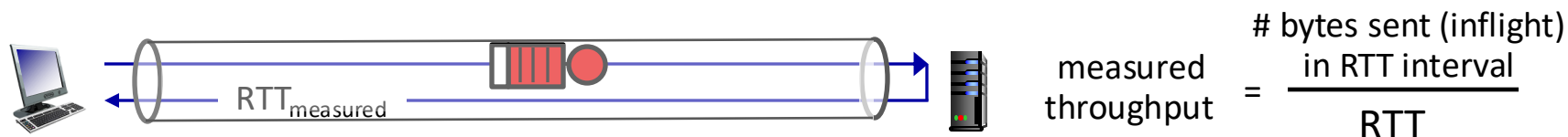
TCP and the congested “bottleneck link”

- multiple flows collectively congest bottleneck link:
 - each executing their own TCP congestion control
 - each of N flows should ideally receive throughput of R/N



Keeping the pipe just full enough

Keeping pipe “just full enough, but no fuller”: keep bottleneck link busy transmitting, but avoid high delays/buffering



Intuition:

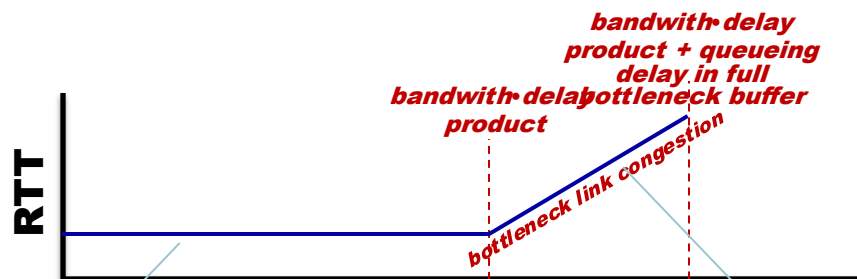
ideal amount of
inflight data for a
connection

n_{inflight}

= *connection's share of bottleneck_bandwidth • min_RTT*

connection's “**bandwidth-delay product**”

Keeping the pipe just full, but no fuller



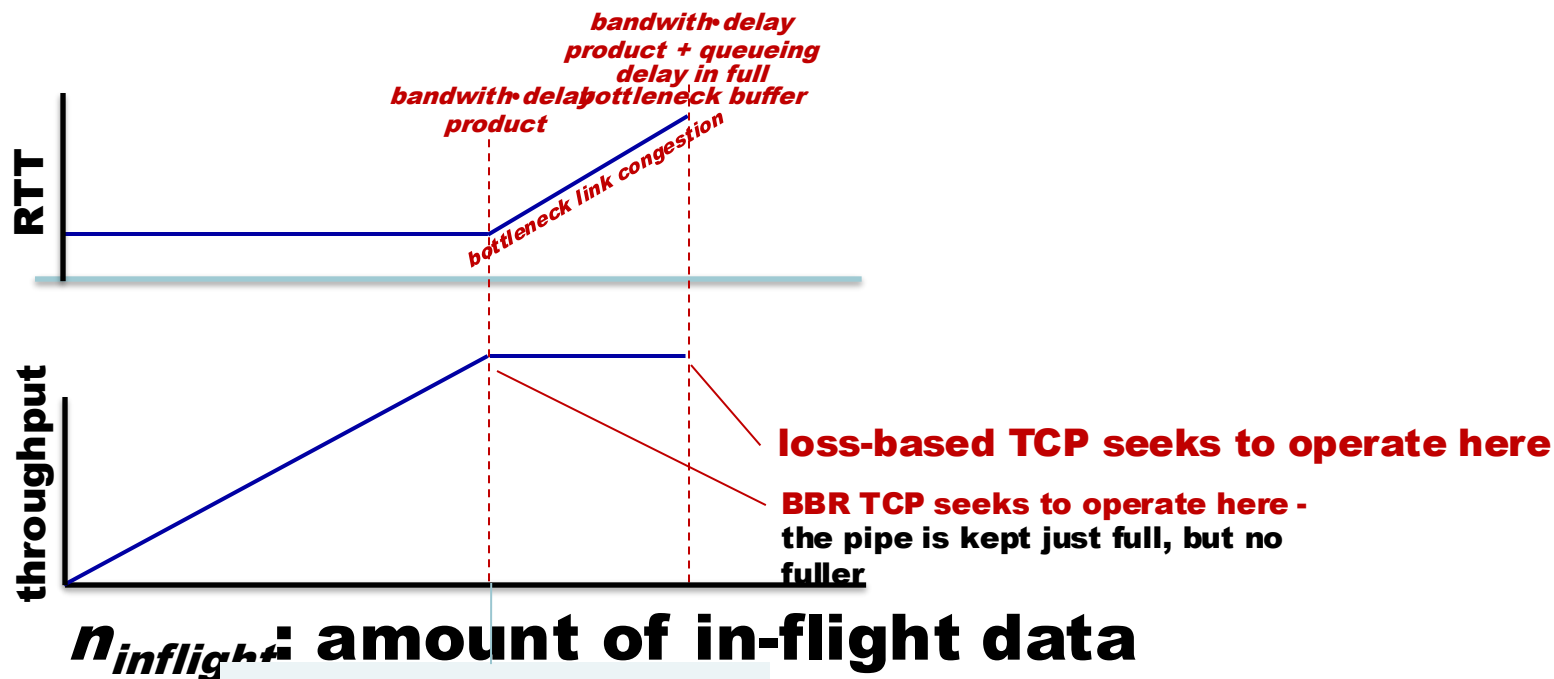
$n_{inflight}$ = amount of in-flight data

increasing $n_{inflight}$ in this region will not increase RTT, since arrival rate of packet data to queue is less than transmission rate

Here, arrival rate of packet data to bottleneck link equals link transmission rate

Increasing $n_{inflight}$ in this region increases RTT since arrival rate of packet data to bottleneck link exceeds link transmission rate

Keeping the pipe just full, but no fuller



$n_{inflight}$: amount of in-flight data

Here, arrival rate of packet data to bottleneck link is equal to link transmission rate

Source-Based Algorithms: TCP Vegas

- Uses **RTT measurements** to detect early congestion
- Calculates:
 - Expected rate = Window size / Base RTT
 - Actual rate = Window size / Current RTT
- If Actual < Expected → congestion building → decrease rate
- Advantages: smoother throughput, fewer losses
- Drawback: can be outperformed by loss-based flows (Reno, CUBIC)

TCP BBR (Bottleneck Bandwidth and RTT)

- Developed by Google (2016)
- Models the network path: estimates **bottleneck bandwidth** and **RTT**
- Sends data at a rate $\approx \text{BottleneckBandwidth} \times \text{RTT}$
- Not loss-driven; focuses on maximizing delivery rate
- Adapts quickly to network changes
- Advantages: higher throughput, lower latency
- Limitation: coexistence issues with loss-based TCPs

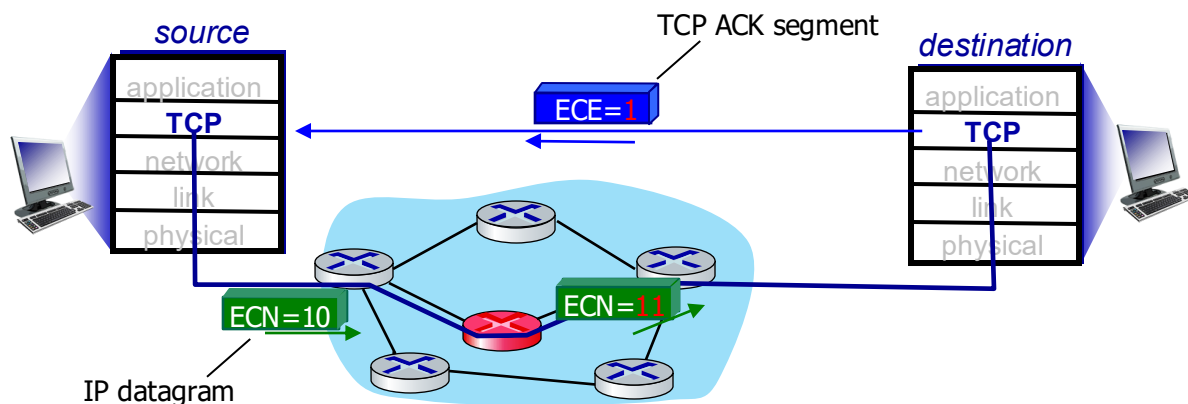
BBR (Bottleneck Bandwidth and RTT)

- regulates $n_{inflight}$
- at longer time intervals:
 - reduces $n_{inflight}$ to drain pipe, measure new min_RTT
- at shorter time intervals:
 - *acceleration*: increases sending rate, $n_{inflight}$, until reaches throughput plateau, (saturating available per-flow link capacity)
 - *cruising*: sends at the rate that network is delivering data, as evidenced through received ACKs
 - *deceleration*: purposefully reduces $n_{inflight}$, decreasing queue pressure, looking for lower min_RTT

Explicit congestion notification (ECN)

TCP deployments often implement *network-assisted* congestion control:

- two bits in IP header (ToS field) marked *by network router* to indicate congestion
 - *policy* to determine marking chosen by network operator
- congestion indication carried to destination
- destination sets ECE bit on ACK segment to notify sender of congestion
- involves both IP (IP header ECN bit marking) and TCP (TCP header C,E bit marking)



DCTCP (Data Center TCP)

- Designed for **low-latency, high-throughput** data centers
- Uses **ECN** to provide fine-grained congestion feedback
- Switch marks packets when queue length $>$ threshold K
- Sender computes fraction α of marked packets and updates window:
- Maintains high throughput with minimal queueing
- Requires ECN support and fair coexistence with classic TCP

DCTCP (Data Center TCP)

■ Mechanism:

- Adapts **ECN**: measures fraction of bytes encountering congestion
- **Sender**: scales congestion window based on fraction of marked bytes
- **Standard TCP** applies if actual packet loss occurs

■ Switch Operation:

- Packet arrives
- Mark packet with CE bit if:

$$K > (RTT \times C)/7$$

- K = queue threshold, C = link rate

DCTCP (Data Center TCP)

- **Receiver Operation:**
- Maintain CE and SeenCE per flow
- On each packet:
 - CE=1 & SeenCE=False → SeenCE=True, send immediate ACK
 - CE=0 & SeenCE=True → SeenCE=False, send immediate ACK
 - Otherwise → continue delayed ACKs every n packets

$$K > (RTT \times C)/7$$

- Finally, sender computes the fraction of bytes that encountered congestion during the previous observation window
- $\text{total bytes transmitted} / \text{the bytes acknowledged with the ECE flag set}$

Final Words...

- DCTCP grows the congestion window in exactly the same way as the standard algorithm
- DCTCP reduces the window in proportion to how many bytes encountered congestion during the last observation window.

CONGESTION AVOIDANCE MECHANISM

Congestion Avoidance Mechanism

- TCP controls congestion after it occurs, not before.
- It gradually increases load until congestion happens, then backs off.
- Congestion avoidance aims to predict congestion and reduce sending rate before packet loss.
- Distinction: Congestion control = reactive, Congestion avoidance = proactive.

Congestion Avoidance Mechanism

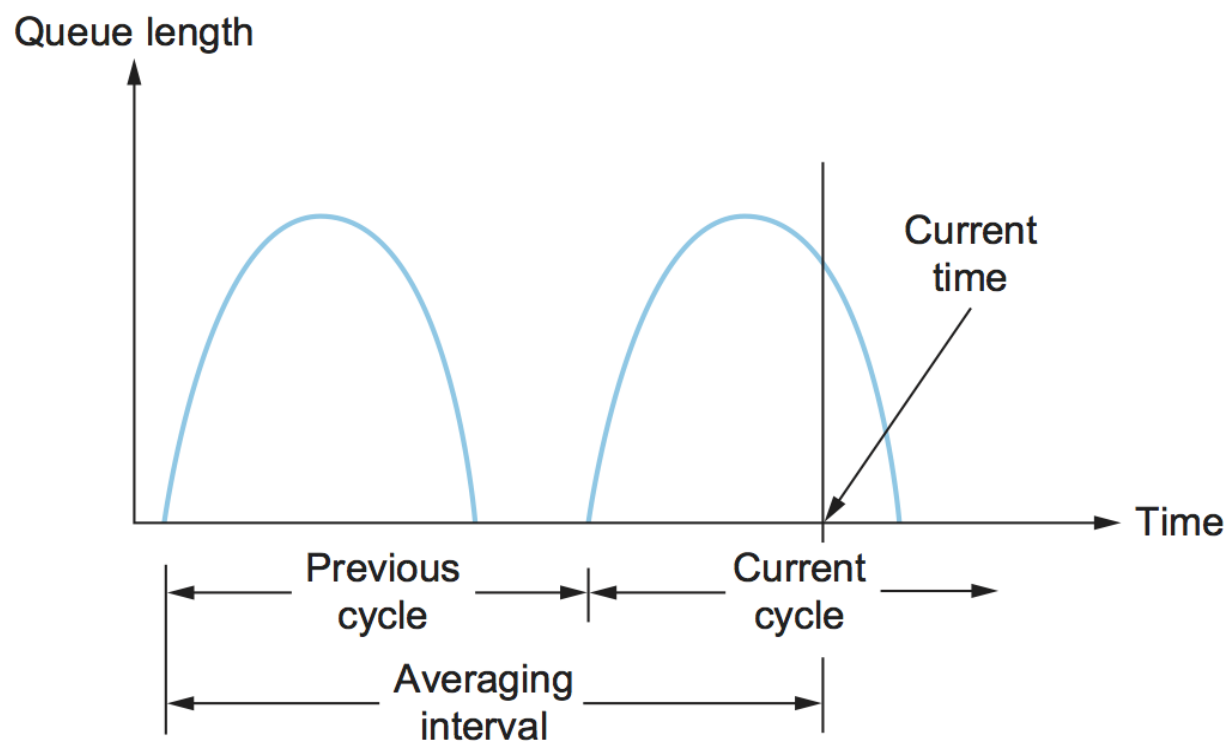
- DEC Bit
 - First developed for Digital Network Architecture (DNA): connectionless network with connection-oriented transport
 - Can also be applied to TCP and IP.
 - Goal: share congestion control responsibility between routers and end nodes.

Congestion Avoidance Mechanism

- DEC Bit
 - Routers **monitor their load** and notify end nodes **before congestion**.
 - Notification via **binary congestion bit** in packets (**DECbit**).
 - Destination copies the congestion bit into **ACKs** back to the source.
 - Source **adjusts sending rate** to **avoid congestion**.

Congestion Avoidance Mechanism

- DEC Bit
 - Single congestion bit added to packet header.
 - Router sets the bit if average queue length ≥ 1 when packet arrives.
 - Average queue length measured over a time interval = last busy + idle cycle + current busy cycle



Queue length at a router as a function of time

Congestion Avoidance Mechanism

- DEC Bit
 - Router computes average queue length by dividing area under the queue curve by the time interval.
 - Queue length = 1 is used to set congestion bit as a trade-off
 - queue length of 1 is a trade-off between significant queuing and increased idle time

Congestion Avoidance Mechanism

- DEC Bit
 - Source tracks packets that triggered the congestion bit.
 - Maintains a congestion window (like TCP).
 - Monitors fraction of packets in the last window that had the bit set.

Congestion Avoidance Mechanism

- DEC Bit
 - **< 50%** of packets with congestion bit → **increase congestion window by 1.**
 - **$\geq 50\%$** of packets with congestion bit → **reduce congestion window to $0.875 \times$ previous value**
 - 50% was chosen as the threshold based on analysis

Congestion Avoidance Mechanism

- DEC Bit
 - 50% threshold corresponds to the peak of the power curve.
 - Additive increase / multiplicative decrease ($1 / 0.875$) ensures stability.

Congestion Avoidance Mechanism

- Random Early Detection (RED)
 - **Random Early Detection (RED):** similar to DECbit, routers monitor their **queue length** and notify source **before congestion**.
 - Invented by **Sally Floyd & Van Jacobson (1990s)**.
 - **Differs from DECbit in two major ways-**
 - *implicitly* notifies the source of congestion by dropping one of its packets
 - Decision on when to drop a packet and what packet to drop

Congestion Avoidance Mechanism

- Random Early Detection (RED)
 - RED **implicitly notifies congestion** by **dropping packets** instead of sending a bit.
 - Source detects congestion via **timeout or duplicate ACKs**.
 - Designed to work with **TCP's packet-loss-based congestion detection**.

Congestion Avoidance Mechanism

- Random Early Detection (RED)
 - RED drops packets early to signal congestion sooner.
 - Early drops help the source slow down before buffer overflows, reducing future packet loss.

Congestion Avoidance Mechanism

- Random Early Detection (RED)
- RED differs from DECbit in **how and when packets are dropped**.
- Uses **early random drop**: packets are dropped with a **probability** once queue exceeds a **threshold**, not only when full.

Congestion Avoidance Mechanism

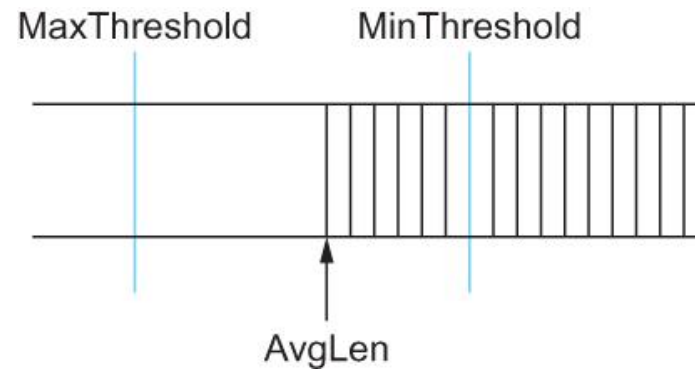
- Random Early Detection (RED)
 - RED computes average queue length (AvgLen) using a weighted running average, similar to TCP timeout calculation
 - $$\text{AvgLen} = (1 - \text{Weight}) \times \text{AvgLen} + \text{Weight} \times \text{SampleLen}$$
 - where $0 < \text{Weight} < 1$ and SampleLen is the length of the queue when a sample measurement is made.
 - **Software:** queue length measured **on each packet arrival**
 - **Hardware:** queue length measured at **fixed intervals**

Congestion Avoidance Mechanism

- Random Early Detection (RED)
 - RED uses **two thresholds**: **MinThreshold** and **MaxThreshold**.
 - Packet handling depends on **current AvgLen** relative to these thresholds.
 - if $\text{AvgLen} \leq \text{MinThreshold}$
 - $\rightarrow$ queue the packet
 - if $\text{MinThreshold} < \text{AvgLen} < \text{MaxThreshold}$
 - $\rightarrow$ calculate probability P
 - $\rightarrow$ drop the arriving packet with probability P
 - if $\text{MaxThreshold} \leq \text{AvgLen}$
 - $\rightarrow$ drop the arriving packet

Congestion Avoidance Mechanism

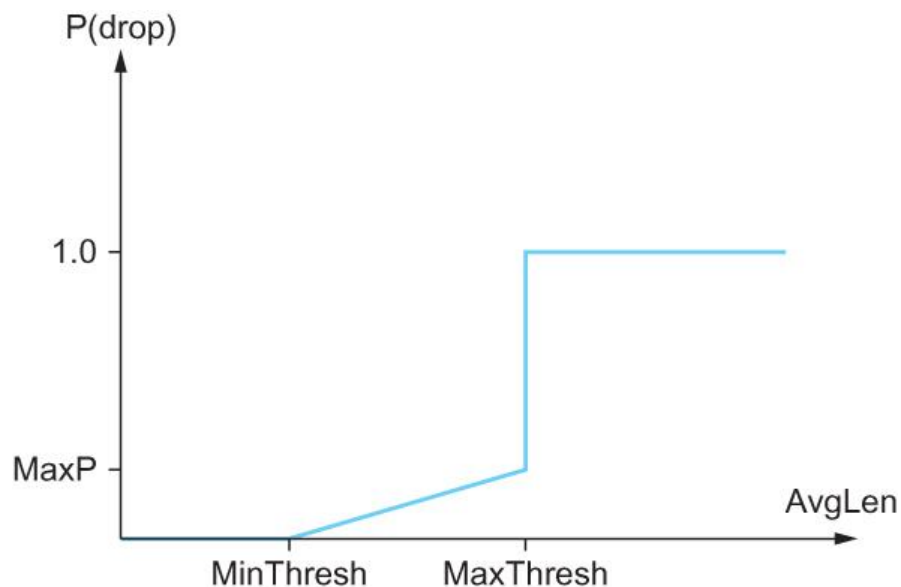
- Random Early Detection (RED)



RED thresholds on a FIFO queue

Congestion Avoidance Mechanism

- Random Early Detection (RED)



Drop probability function for RED

Congestion Avoidance Mechanism

- Random Early Detection (RED)
 - P is a function of both AvgLen and how long it has been since the last packet was dropped.
 - Specifically, it is computed as follows:
 - $\text{TempP} = \text{MaxP} \times (\text{AvgLen} - \text{MinThreshold}) / (\text{MaxThreshold} - \text{MinThreshold})$
 - $P = \text{TempP} / (1 - \text{count} \times \text{TempP})$
 - TempP is the variable that is plotted on the y-axis
 - Count keeps track of how many newly arriving packets have been queued (not dropped)
 - P increases slowly as count increases, thereby making a drop increasingly likely as the time since the last drop increases

ECN (Explicit Congestion Notification)

- Rather than dropping, routers mark packets when congestion is imminent
- Receivers echo the marking back to senders
- Sender reduces rate based on marking, not loss
- Helps avoid packet loss while controlling congestion

QUALITY OF SERVICE

Quality of Service

- Packet-switched networks **can support multimedia** (audio, video, data).
- Digitized audio/video are just **bit streams**.
- Past obstacle: **high bandwidth requirements**.
- Recent advances: **better coding + faster links** reduce bandwidth constraints.

Quality of Service

- **Bandwidth alone** is not enough for audio/video transmission.
- Real-time interaction (e.g., telephone conversation) requires **minimal delay**.
- Applications sensitive to delivery timeliness are called **real-time applications**.

Quality of Service

- **Voice and video** are typical real-time applications.
- Other examples: **industrial control** (e.g., robot commands) and **time-sensitive file transfers** (e.g., overnight database updates).

Quality of Service

- Real-time applications need **assurance of timely delivery**.
- End-to-end retransmission ensures correctness, **not timeliness**.
- Network must **treat some packets differently**, unlike best-effort.
- Networks providing **differentiated service** support **Quality of Service (QoS)**.

Quality of Service

- Real-Time Applications
 - Data is generated by collecting samples from a microphone and digitizing them using an A $\rightarrow$ D converter
 - The digital samples are placed in packets which are transmitted across the network and received at the other end
 - At the receiving host the data must be played back at some appropriate rate

Quality of Service

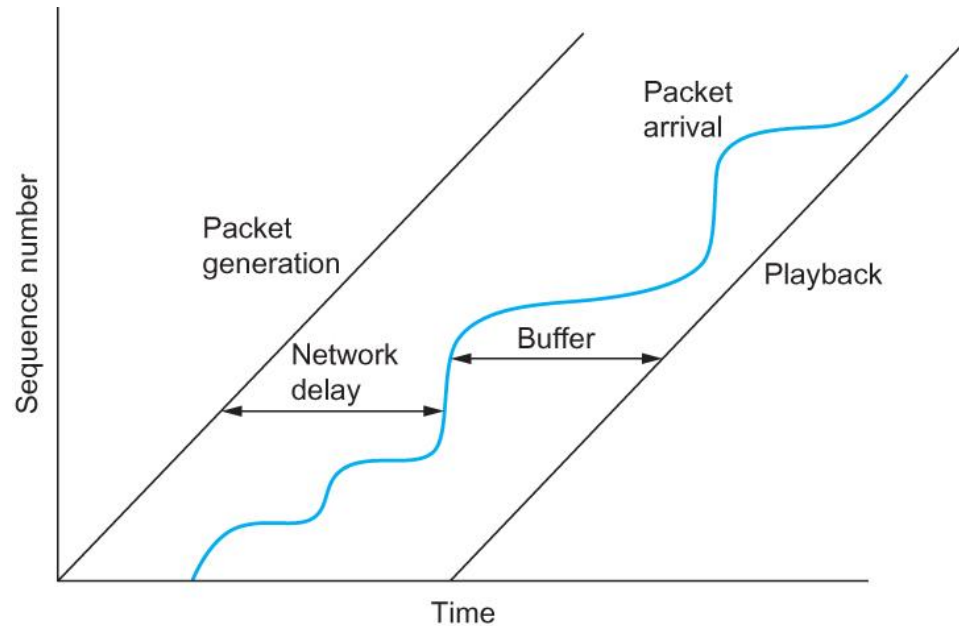
- **Real-Time Applications:**
 - Voice samples collected every **125 μ s** must be **played back at the same rate**.
 - Each sample has a **specific playback time** at the receiver.
 - Playback time for each sample is **125 μ s after the previous one**.
 - **Late arrival** renders data **useless**.

Quality of Service

- **Real-Time Applications:**
 - Some audio apps have **strict playback delay limits**.
 - Conversation becomes difficult if **delay > 300 ms**.
 - Network should **guarantee delivery within 300 ms**.
 - Early-arriving data is **buffered until playback**.

Quality of Service

■ Real-Time Applications



A playback buffer

Quality of Service

- **Taxonomy of Real-Time Applications:**
 - Applications can be categorized by **tolerance to data loss**.
 - Loss may occur from **late arrival** or network issues.
 - **Small losses** (e.g., one audio sample) have **minimal quality impact**.
 - **High loss** leads to **unintelligible speech or degraded quality**.

Quality of Service

- **Taxonomy of Real-Time Applications:**
 - Some applications, like **robot control**, **cannot tolerate any loss**.
 - Real-time applications are classified as:
 - **Loss-tolerant** – occasional packet loss acceptable.
 - **Loss-intolerant** – packet loss is unacceptable.

Quality of Service

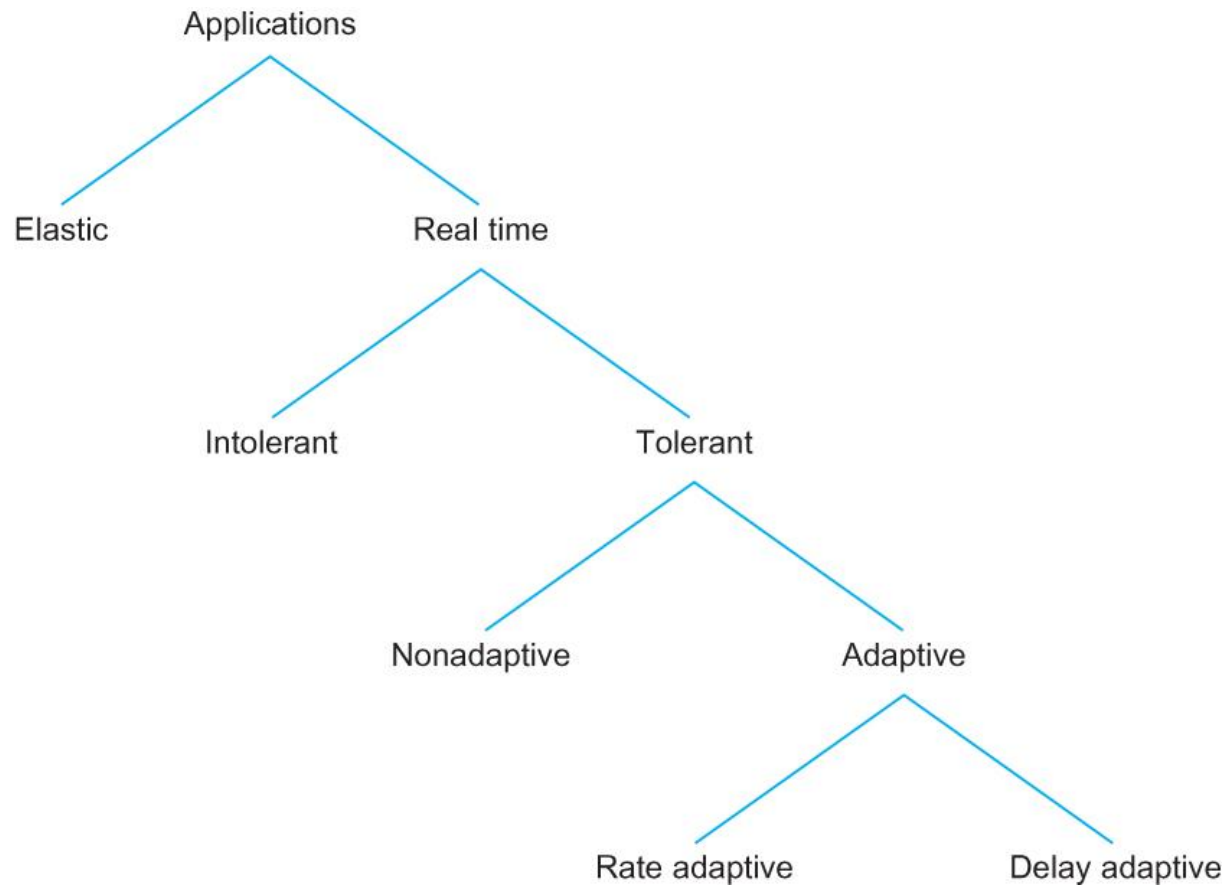
- **Taxonomy of Real-Time Applications – Adaptability:**
 - Some applications can **adapt to network delay**.
 - Example: Audio playback **buffers packets** based on observed arrival times.
 - Adjusting playback point (e.g., from 300 ms → 100 ms) can **improve user experience**.
 - **Faster playout temporarily** may be needed to catch up.

Quality of Service

- **Taxonomy of Real-Time Applications – Adaptability:**
 - **Delay-adaptive:** adjust **playback point** based on network delay.
 - **Rate-adaptive:** adjust **bit rate vs. quality** based on available bandwidth.
 - Quality can be **increased** if more bandwidth becomes available.

Quality of Service

■ Taxonomy of Real-Time Applications



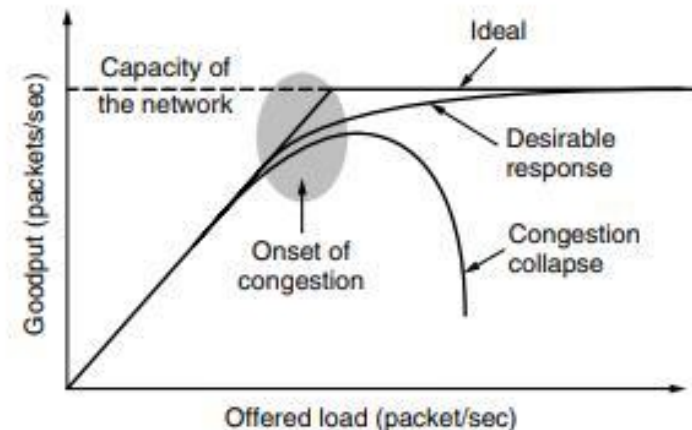
Quality of Service

- **Approaches to QoS Support:**
 - **Fine-grained:** QoS for individual applications/flows (e.g., **Integrated Services + RSVP**).
 - **Coarse-grained:** QoS for traffic classes/aggregates (e.g., **Differentiated Services**, widely deployed).

What is congestion?

Impact of congestion

- Packet queues at links start to grow
- Packets start dropping
- Sources start re-transmitting
- After a while only re-transmissions occupy the network
- Network resources start getting utilized in useless work (packets in queues that get timed out and re-transmitted)
- "Goodput" goes to nearly zero



Congestion control:

- What is congestion control?
 - ✓ making sure the network is able to carry the offered traffic.
- Two solutions come to mind:
 - Increase the resources
 - Decrease the load
- Best way to handle congestion:

The host can get a “slow down” message either because the receiver cannot handle the load or because the network cannot handle it.

Approaches to congestion control

- The most basic way to avoid congestion is to build a network that is well matched to the traffic that it carries.
- Other ways to control congestion:

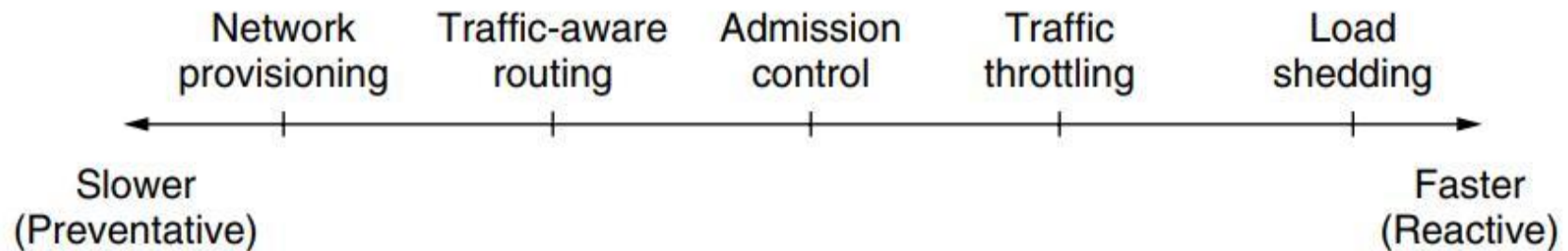


Fig.2 Timescales of approaches to congestion control

Network Provisioning:

- Dynamic allocation of resources, such as routers, links, and bandwidth, to adapt to congestion or upgrade heavily utilized components, driven by long-term traffic patterns.
- Ensures fault tolerance and optimal performance in a network infrastructure.

Traffic Aware Routing:

- Dynamic strategy that adapts network routes based on real-time traffic conditions
- Utilizes methods like adjusting path weights and incorporating live updates
- Enables efficient data transmission and avoids congestion hotspots.

Admission Control:

- Sometimes it is not possible to increase capacity.
- Only way is to decrease the load.
- In a virtual-circuit network, new connections can be refused if they would cause the network to become congested.

Traffic Throttling

- When congestion is imminent the network can deliver feedback to the sources whose traffic flows are responsible for the problem.
- The network can request these sources to throttle their traffic, or it can slow down the traffic itself.

Load Shedding:

- When routers are being inundated by packets that they cannot handle, they just throw them away.

Quality of Service

- **Quality of service** mechanisms let a network with less capacity meet application requirements just as well at a lower cost.

Four issues must be addressed to ensure quality of service:

1. What applications need from the network.
2. How to regulate the traffic that enters the network.
3. How to reserve resources at routers to guarantee performance.
4. Whether the network can safely accept more traffic.

Modules of Quality of Service

- Application Requirements
- Traffic Shaping
- Packet Scheduling
- Integrated Service
- Differentiated Service

Application Requirements:

Application	Bandwidth	Delay	Jitter	Loss
Email	Low	Low	Low	Medium
File sharing	High	Low	Low	Medium
Web access	Medium	Medium	Low	Medium
Remote login	Low	Medium	Medium	Medium
Audio on demand	Low	Low	High	Low
Video on demand	High	Low	High	Low
Telephony	Low	High	High	Low
Videoconferencing	High	High	High	Low

Fig. Stringency of applications' quality-of-service requirements

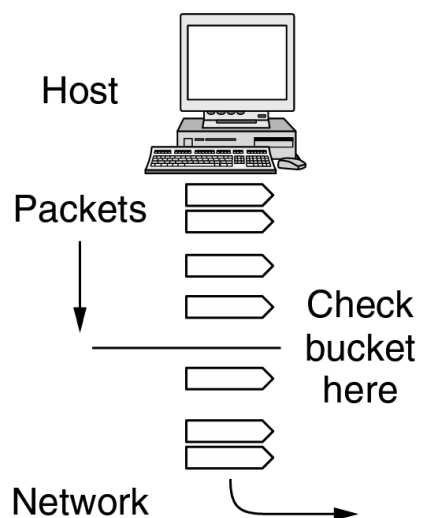
- To accommodate a variety of applications, networks may support different categories of QoS.
 1. Constant bit rate (e.g., telephony).
 2. Real-time variable bit rate (e.g., compressed videoconferencing).
 3. Non-real-time variable bit rate (e.g., watching a movie on demand).
 4. Available bit rate (e.g., file transfer).

Traffic Shaping

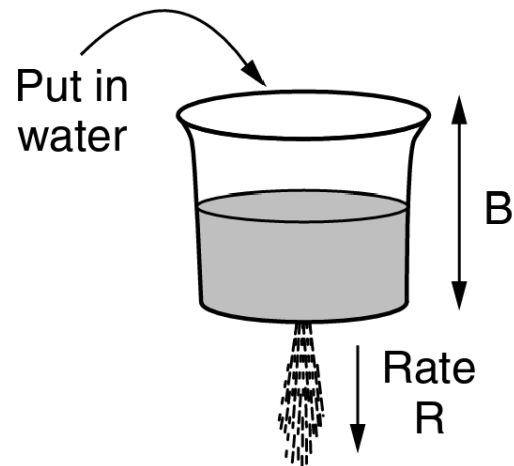
- *Traffic shaping is a technique for regulating the average rate and burstiness of a flow of data that enters the network.*
- The goal is to allow applications to transmit a wide variety of traffic that suits their needs, including some bursts
- Have a simple and useful way to describe the possible traffic patterns to the network
- Traffic shaping reduces congestion and thus helps the network live up to its promise

Traffic shaping algorithms:

- Leaky Bucket
- Token Bucket



a) Leaky bucket with Packet



b) Leaky bucket with water

Leaky Bucket

- The leaky bucket algorithm is used to control rate in a network.
- It is implemented as a single server queue with content service time.
- If the bucket overflows, then packets are discarded.
- In the algorithm the input rate can vary but the output rate remains constant.
- The algorithm serves bursty traffic into fixed rate traffic by averaging the data rate.

Algorithm

Step1: Initialize a counter to n at the tick of the clock.

Step2: Repeat until n is smaller than the packet size of the packet at the head of the queue.

- Pop a packet out of the head of the queue, say P .
- Send the packet P , into the network
- Decrement the counter by the size of packet P .

Step3: Reset the counter and go to step 1.

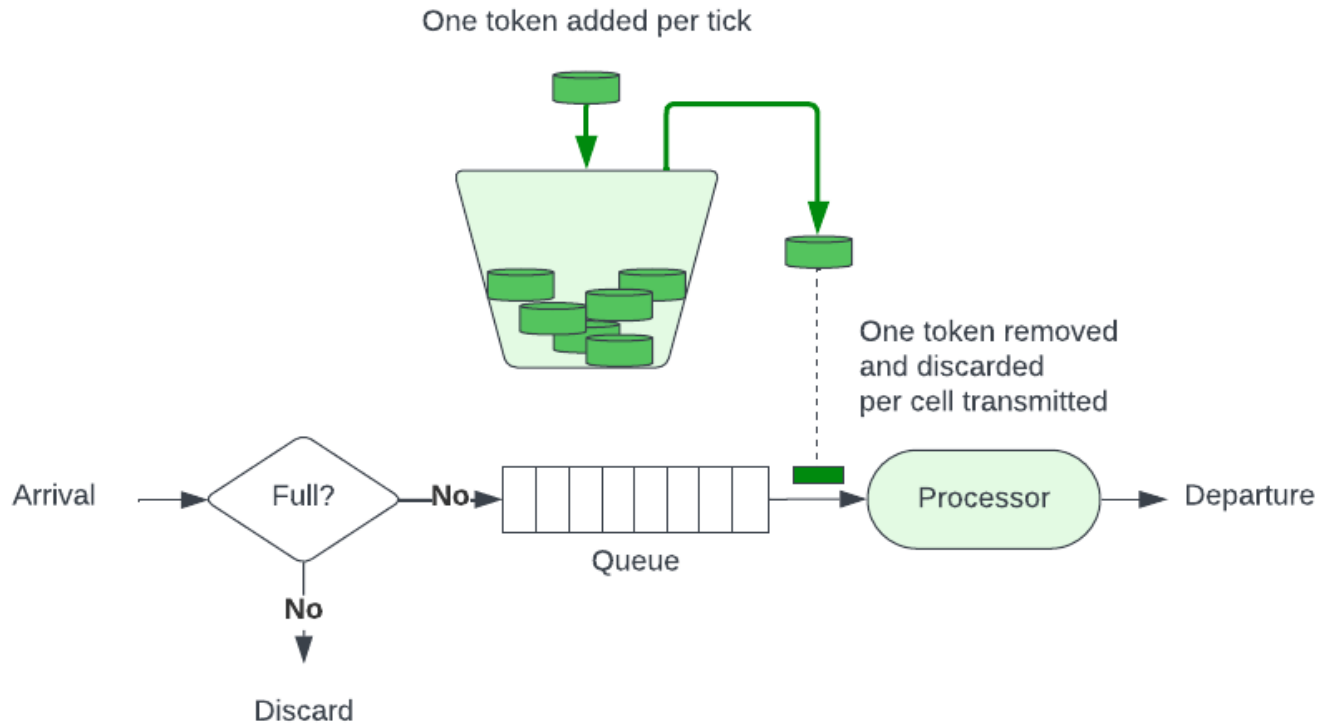
Token Bucket Algorithm

- Used for traffic shaping and rate limiting
- Considers:
 - maximum capacity or permission available to the system
 - controls the amount of data transmitted over time
 - flexibility for bursts when tokens are available

Token Bucket

- When an event occurs, it requests a token from the bucket.
- If a token is available, it is removed from the bucket, allowing the event to occur.
- If no tokens are available, the event is blocked or delayed until a token becomes available.
- After each event, the algorithm checks whether the bucket has exceeded its capacity, and if so, additional tokens are discarded.
- This ensures that the bucket is not too full and the system remains controlled.

Token Bucket Algorithm



Algorithm:

- Step1: A token is added every Δt time.
- Step2: The bucket can hold at most b -token. If token arrive when bucket is full it is discarded.
- Step3: When packet of m bytes arrive, m tokens are removed from the bucket and the packet is sent to the network.
- Step4: If less, then n tokens are available, no tokens are removed from the bucket and the packet is considered to be non conformant.
- Step5: The conformant packet may be enqueued for subsequent transmissions when sufficient tokens have been accumulated in the buffer.

Advantages of Token Bucket over Leaky Bucket

- **Token Handling:** When the bucket is full, excess tokens are discarded, not packets. In contrast, leaky bucket discards packets when overflow occurs.
- **Support for Bursts:** Token bucket allows large bursts of data at high speed, while leaky bucket enforces a constant transmission rate.
- **Traffic Shaping Control:** It offers predictable shaping by adjusting token arrival rate and maximum token count
- **Quality of Service (QoS):** Different token rates can prioritize various traffic types, enabling better QoS
- **Efficiency for High-Speed Data:** Suitable for streaming and high-throughput transfers, as bursts are transmitted efficiently.

Disadvantages of Token Bucket Algorithm

- **Token Accumulation:** Tokens are generated at a fixed rate even when traffic is absent, leading to buildup and inefficient usage.
- **Increased Latency:** If the bucket is empty, packets must wait for new tokens, causing delay and possible packet loss.
- **Complex Implementation:** Managing different token generation rates for multiple traffic types increases configuration difficulty.
- **Inefficient with Large Bursts:** Allowing sudden bursts of data may waste bandwidth and cause congestion, unlike the leaky bucket which enforces smoother traffic flow.

Packet Scheduling

- Packet scheduling algorithms allocate bandwidth and other router resources by determining which of the buffered packets to send on the output line next.
- Three different kinds of resources can potentially be reserved for different flows:
 - 1. Bandwidth. 2. Buffer space. 3. CPU cycles.

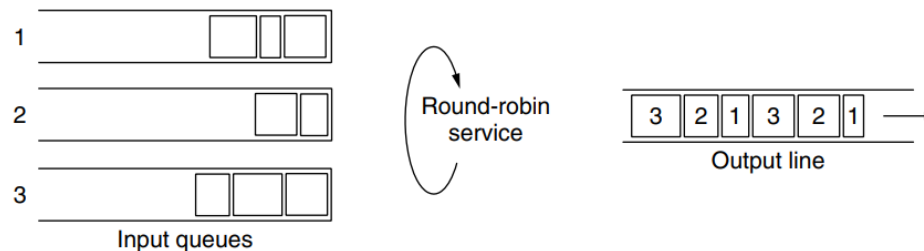
Types of Packet Scheduling Algorithms

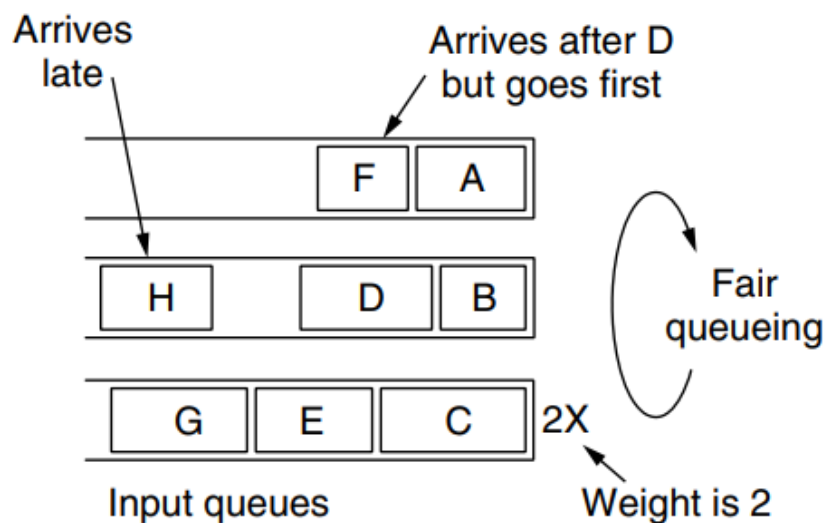
1) **FIFO:**

- Router buffers packets in a queue for each output line until they can be sent, and they are sent in the same order that they arrived.
- FIFO routers usually drop newly arriving packets when the queue is full. Since the newly arrived packet would have been placed at the end of the queue, this behavior is called tail drop.

2) Round-Robin Fair Queueing

- The essence of this algorithm is that routers have separate queues, one for each flow for a given output line.
- When the line becomes idle, the router scans the queues round-robin. It then takes the first packet on the next queue. In this way, with n hosts competing for the output line, each host gets to send one out of every n packets.
- It is fair in the sense that all flows get to send packets at the same rate.





(a)

Packet	Arrival time	Length	Finish time	Output order
A	0	8	8	1
B	5	6	11	3
C	5	10	10	2
D	8	9	20	7
E	8	8	14	4
F	10	6	16	5
G	11	10	19	6
H	20	8	28	8

(b)

Fig. (a) Weighted Fair Queueing. (b) Finishing times for the packets.

Integrated Services

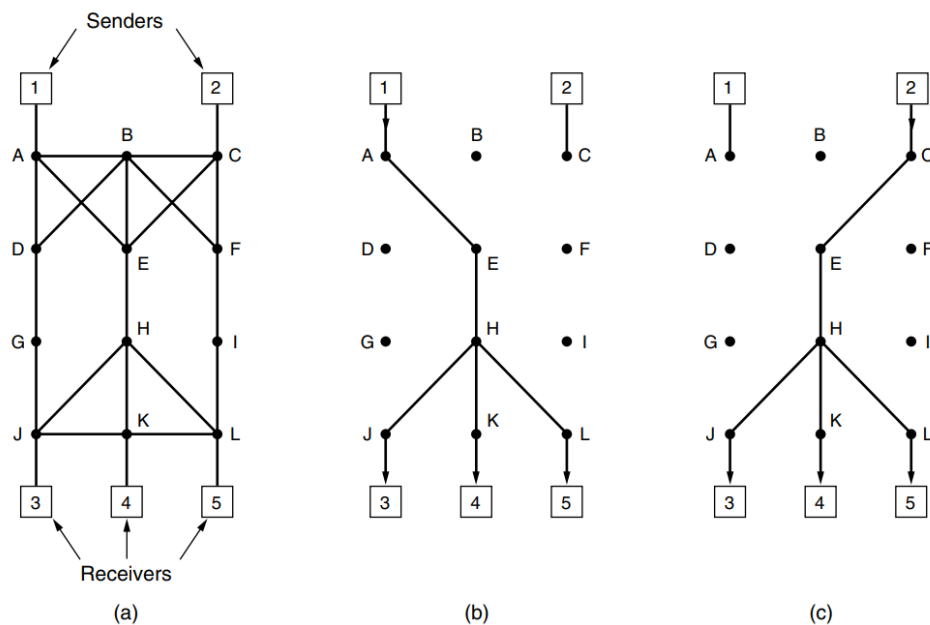
- Integrated service provide the capability for IP applications to request and reserve bandwidth using the ReSerVation Protocol (RSVP) and quality of service (QoS) APIs.
- **RSVP—The Resource reSerVation Protocol:** This protocol is used for making the reservations; other protocols are used for sending the data.
- Integrated service policies use the RSVP and the Resource Reservation Setup Protocol API (RAPI) to guarantee an end-to-end connection.
- Complex service.

RSVP

- Allows multiple senders to transmit to multiple groups of receivers
- Permits individual receivers to switch channels freely
- Optimizes bandwidth use while eliminating congestion

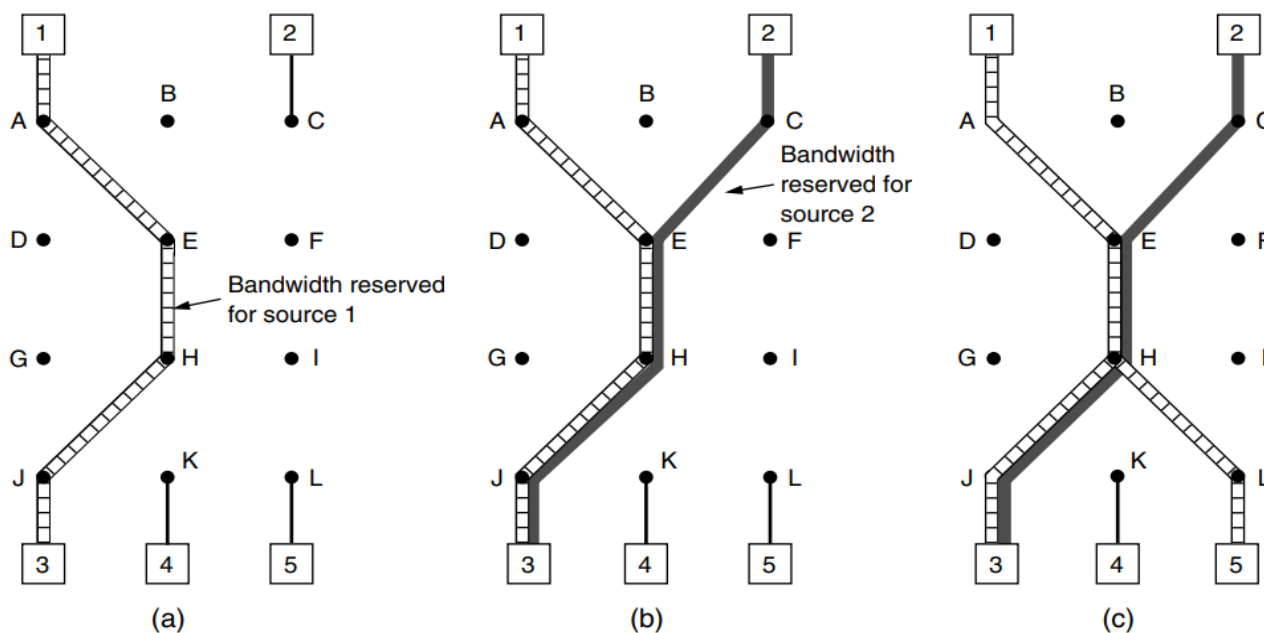
Example:

Hosts 1 and 2 are multicast senders, and hosts 3, 4, and 5 are multicast receivers. In this example, the senders and receivers are disjoint, but in general, the two sets may overlap.



(a) A network. (b) The multicast spanning tree for host 1. (c) The multicast spanning tree for host 2.

- a) A- Host 3 requests a channel to Host 1
- b) Once reservation is done, packets can flow from 1 to 3 without congestion
- c) B- Host 3 next reserves a channel to Host 2, so that users can watch both contents at the same time. A second path is reserved.
- d) C- Host 5 decides to watch content from Host 1. Reservation done only upto router H. Rest, already done.



Differentiated Services

- A simpler approach to quality of service.
- Largely implemented locally in each router without advance setup and without having the whole path involved.
- This approach is known as **class-based** (as opposed to flow-based) quality of service.
- Differentiated services can be offered by a set of routers forming an administrative domain.
- The administration defines a set of service classes with corresponding forwarding rules.
- If a customer subscribes to differentiated services, customer packets entering the domain are marked with the class to which they belong.
- IETF has defined some network-independent service classes.

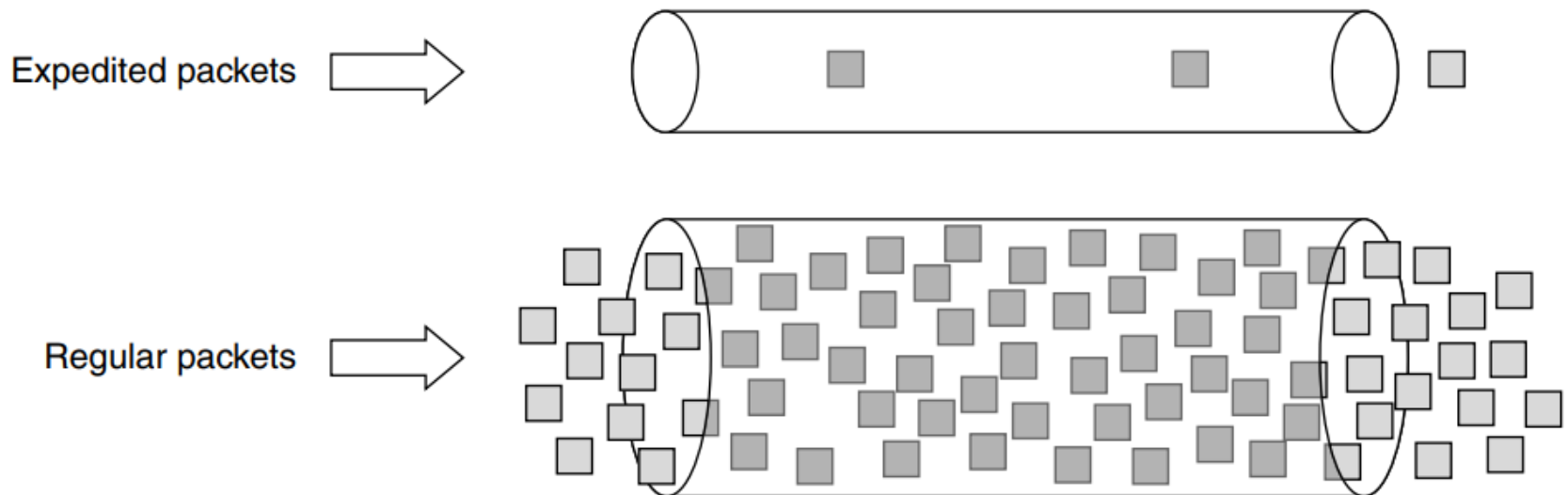
Expedited Forwarding

- Two classes of services are available: regular and expedited.
- Majority of the traffic is expected to be regular.
- Limited fraction of the packets are expedited.
- Expedited packets should be able to transit the network as though no other packets were present.
- They will get low loss, low delay, and low jitter service (just what is needed for VoIP !!)

Expedited Forwarding

Steps

- Packets are classified as expedited or regular and marked accordingly (might be done at the sending host or ingress router)
- Routers may have two output queues- one for regular and other for expedited traffic
- Higher priority given to expedited traffic by the scheduler

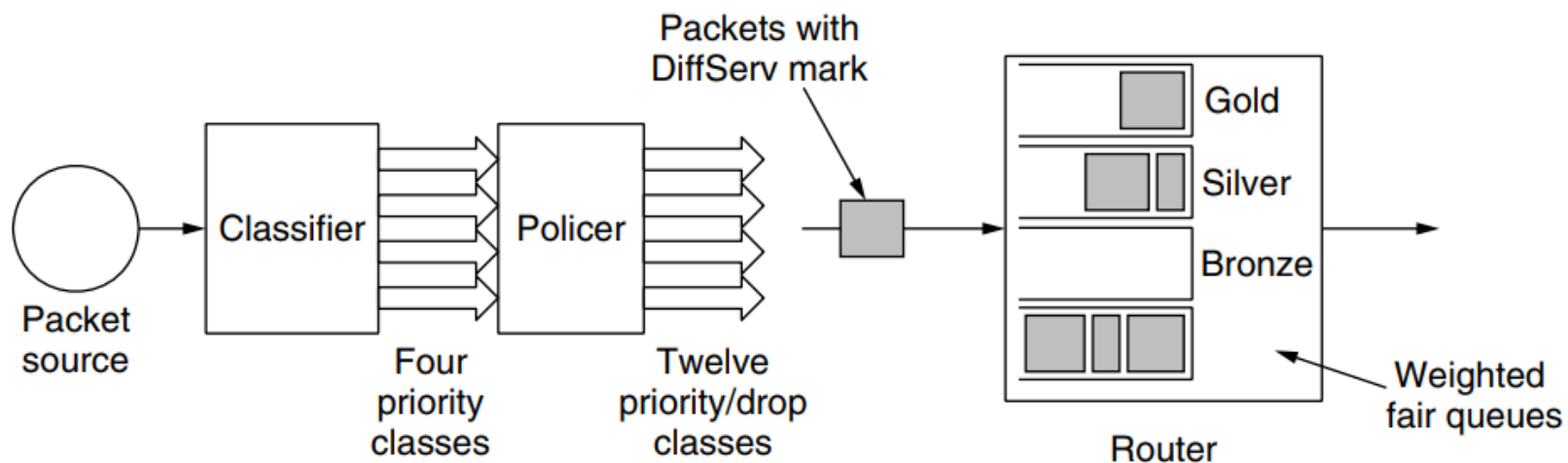


Expedited packets experience a traffic-free network

Assured Forwarding

- Assured forwarding specifies that there shall be four priority classes, each class having its own resources.
- The top three classes might be called *gold, silver, and bronze*.
- In addition, it defines three discard classes for packets that are experiencing congestion: *low, medium, and high*.
- Taken together, these factors define 12 service classes.

A possible implementation of assured forwarding



Quality of Service

- **Integrated Services (IntServ + RSVP):**
 - Developed by **IETF (1995–97)**.
 - Specifies **service classes** for different application types.
 - **RSVP** used to **reserve resources** according to these service classes.

Quality of Service

- **Integrated Services – Service Classes:**
 - **Guaranteed Service:** Network guarantees **maximum packet delay**.
 - **Controlled Load Service:** Provides an **emulated lightly loaded network** for requesting applications, even under heavy overall load.

Quality of Service

- **Integrated Services (RSVP) – Mechanisms:**
 - **Flowspec:** Specifies the **type of service** required, not just destination.
 - **Admission Control:** Network decides **if it can provide the requested service**.
 - **Resource Reservation:** Exchanges **service requests, flowspecs, and admission decisions** between users and network.

Quality of Service

- **Integrated Services (RSVP) – Packet Scheduling:**
 - After **flows and admission control** are defined, switches/routers must **meet flow requirements**.
 - **Packet scheduling** manages how packets are **queued and transmitted** to satisfy these requirements.

Quality of Service

- **Integrated Services (RSVP) – Flowspec:**
 - **Flowspec** has two parts:
 - **TSpec:** Describes **traffic characteristics**.
 - **RSpec:** Describes **requested service**.
 - **Controlled Load:** RSpec is simple, just requests the service.
 - **Guaranteed Service:** RSpec can specify **delay target or bound**.

Quality of Service

- **Integrated Services (RSVP) – TSpec:**
 - Provides the network with **bandwidth info** for **admission control**.
 - Bandwidth **varies over time**, not a single number.
 - Example: Video streams generate **more bits during rapid scene changes**.
 - **Long-term average bandwidth** is insufficient for real-time decisions.

Quality of Service

- **Integrated Services (RSVP) – Flow Example:**
 - 10 flows share a **10 Mbps link**, each expected $\leq$ 1 Mbps $\rightarrow$ **no problem**.
 - **Variable-bit applications** (e.g., compressed video) may **exceed average rate**.
 - If many flows exceed averages, **total arrival rate $>$ link capacity**.
 - Excess data is **queued**, and **queue length grows** the longer the condition persists.

Quality of Service

- **Integrated Services (RSVP) – Token Bucket Filter (TSpec):**
- Describes **bandwidth characteristics** using:
 - **Token rate (r)**
 - **Bucket depth (B)**
- **Tokens** required to send data:
 - 1 token per byte, n tokens per packet of length n
- Tokens **accumulate at rate r** , up to a **maximum of B** .

Quality of Service

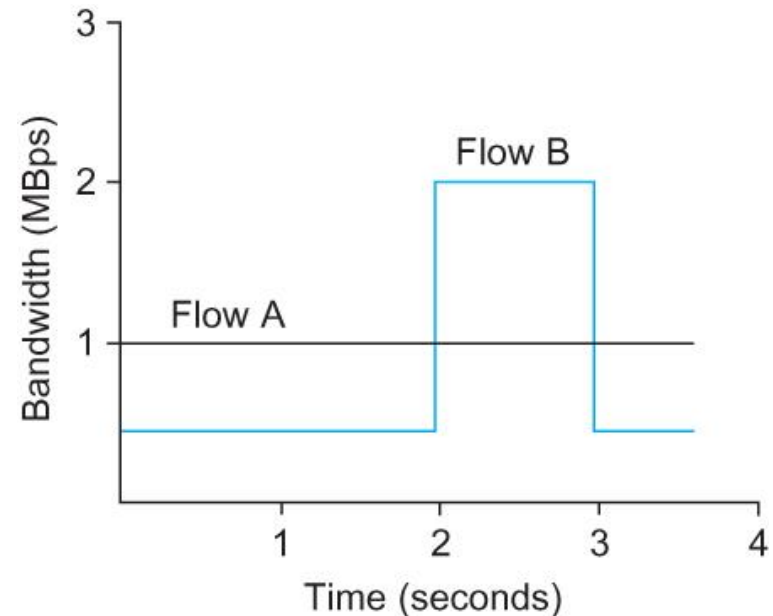
- **Integrated Services (RSVP) – Token Bucket Behavior:**
 - Can send a **burst of up to B bytes** instantly.
 - Over long intervals, **average rate $\leq r$ bytes/sec.**
 - Crucial for **admission control** to determine if new service requests can be accommodated.

Quality of Service

■ Flowspec

·Integrated Services (RSVP) – Token Bucket Example:

- Flow A sends data at **1 MBps**.
- Token Bucket parameters: **$r = 1$ MBps, $B = 1$ byte**.
- Tokens accumulate at **1 MBps**, **maximum 1 token** → **spent immediately**.
- Characterizes **steady-rate flow** for admission control.

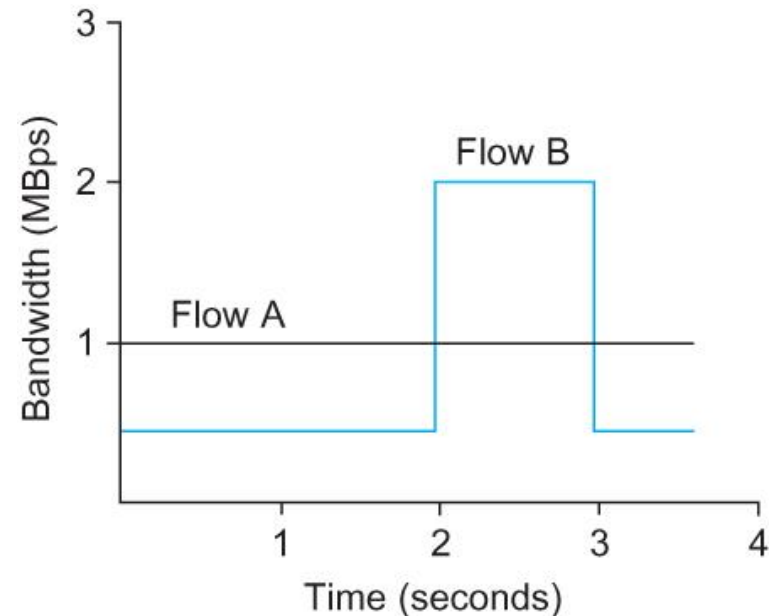


Quality of Service

■ Flowspec

·Integrated Services (RSVP) – Token Bucket Example (Flow B):

- Flow B **averages 1 MBps**:
0.5 MBps for 2 s, 2 MBps for 1 s.
- Token Bucket rate $r = 1$ MBps (long-term average).
- Requires **bucket depth $B \geq 1$ MB** to **store tokens** when sending below average and use them during bursts.

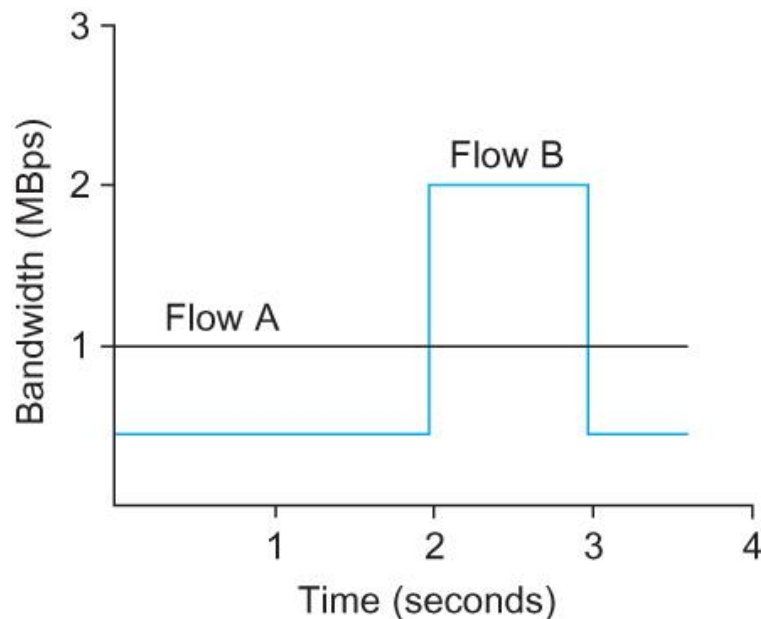


Quality of Service

■ Flowspec

• Integrated Services (RSVP) – Token Bucket Example (Flow B Burst):

- First 2 s: receives **1 MBps** tokens, sends **0.5 MBps** → **saves 1 MB**.
- 3rd second: uses saved **1 MB** of **tokens** to send at **2 MBps**.



Quality of Service

- **Integrated Services (RSVP) – Admission Control:**
 - Checks **TSpec** and **RSpec** of a new flow.
 - Determines if network can **provide requested service** without **degrading existing flows**.
 - **Admit flow** if possible; **deny** if not.

Quality of Service

- **Integrated Services (RSVP) – Reservation Protocol:**
 - Connectionless networks **lack setup protocols** for real-time service.
 - Real-time service requires **more information** to be conveyed to the network.
 - **RSVP** is the main protocol used for **reserving resources** in the Internet.

Quality of Service

- **Integrated Services (RSVP) – Key Principle:**
 - RSVP **preserves robustness** of connectionless networks.
 - Routers can **crash or reboot**, links may change, yet connectivity remains.
 - Achieves this via **soft state** in routers.

Quality of Service

- **Integrated Services (RSVP) – Multicast Support:**
 - RSVP supports **multicast flows** as effectively as unicast.
 - For **one sender** → **one receiver**, two steps are needed **before a receiver can reserve resources**.

Quality of Service

Integrated Services (RSVP) – Reservation Steps:

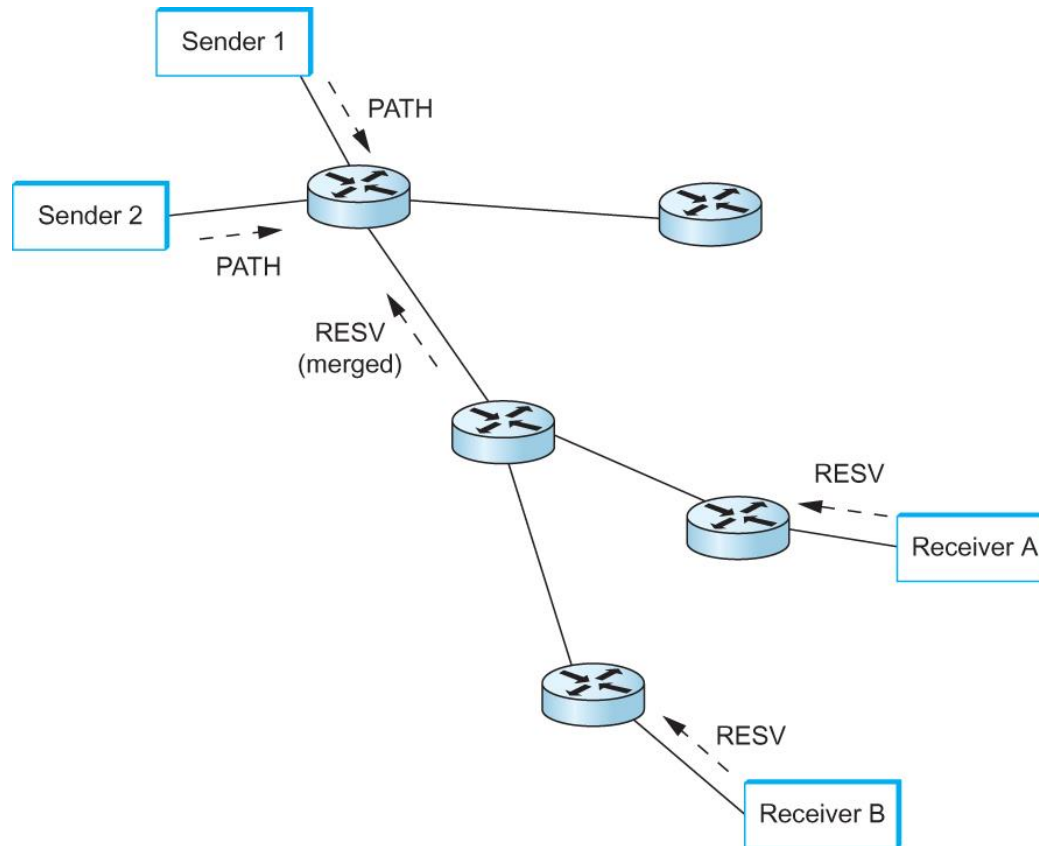
1. Receiver must know **sender's TSpec** to make an appropriate reservation.
2. Receiver must know the **path packets take** to reserve resources at each router.
 - Achieved via a **PATH message** from sender → receiver.
 - Routers use PATH message to determine **reverse path** for reservation messages.

Quality of Service

- **Integrated Services (RSVP) – RESV Message:**
 - Receiver sends **RESV message** back up the multicast tree.
 - Contains **sender's TSpec** and receiver's **RSpec**.
 - Routers allocate **resources** to satisfy the reservation.
 - If successful, RESV propagates; otherwise, **error returned** to receiver.
 - **RESV messages repeated every ~30 s** to maintain reservation.

Quality of Service

- Integrated Services (RSVP)
 - Reservation Protocol



Making reservations on a multicast tree

Quality of Service

- **Integrated Services (RSVP) – Packet Handling:**
 - **Packet Classification:** Associate each packet with the **correct reservation**.
 - **Packet Scheduling:** Manage queues so packets receive the **requested service**.

Quality of Service

- **Differentiated Services (DiffServ):**
 - Allocates resources to **traffic classes**, not individual flows.
 - Some approaches divide traffic into as few as **two classes**.

Quality of Service

- **Differentiated Services – Packet Identification:**
 - Introduce a “**premium**” **class** alongside best-effort.
 - Packets **self-identify** using a **header bit**:
 - 1 → premium
 - 0 → best-effort
 - Avoids the need for **per-flow signaling** like RSVP.

Quality of Service

- **Differentiated Services – Key Questions:**
 - **Who sets the premium bit and when?**
 - **How should routers treat** packets with the premium bit set?

Quality of Service

- **Differentiated Services – Setting the Premium Bit:**
 - Commonly set at an **administrative boundary** (e.g., ISP edge router).
 - Example: Packets from a **paying company** are marked premium.
 - Allows differentiation based on **service level agreements**.

Quality of Service

- **Differentiated Services – Router Handling:**
 - Routers apply **Per-Hop Behaviors (PHBs)** to marked packets.
 - PHBs define **individual router actions**, not end-to-end services.
 - Standardized by the **IETF**.

Quality of Service

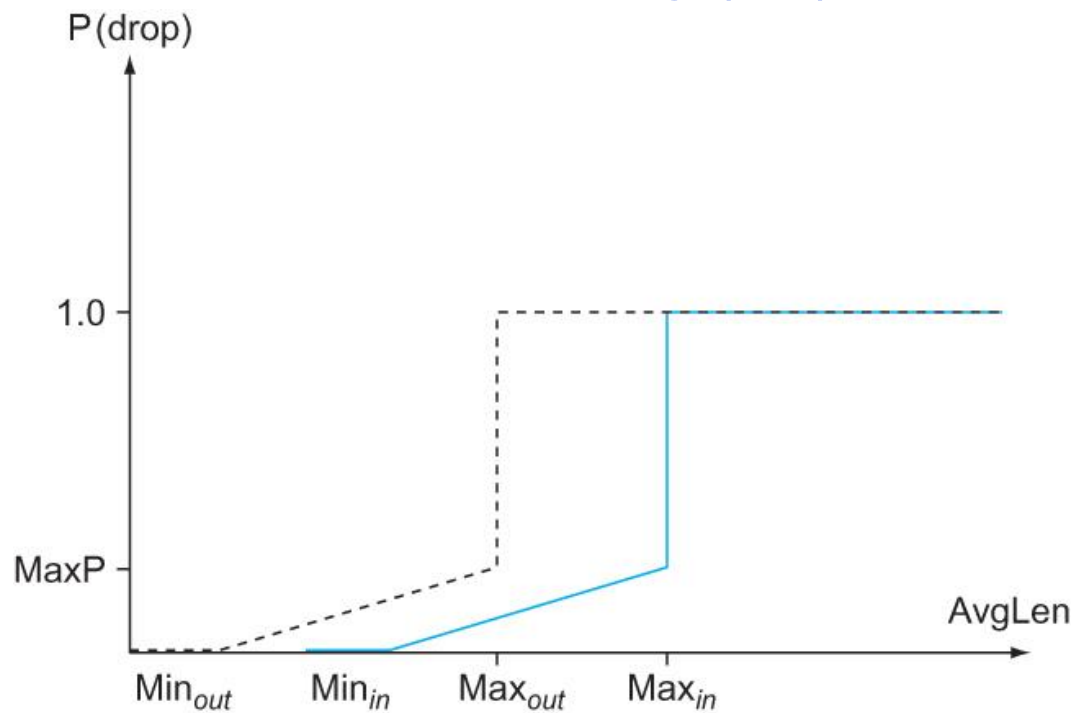
- **Differentiated Services – Expedited Forwarding (EF) PHB:**
 - EF packets are forwarded with **minimal delay and loss**.
 - Router can guarantee this only if **EF arrival rate $\leq$ forwarding capacity**.

Quality of Service

- **Differentiated Services – Assured Forwarding (AF) PHB:**
 - Based on **RED with In and Out (RIO) / Weighted RED**.
 - Two traffic classes: “**in**” and “**out**”.
 - **Out packets** are dropped first under **low congestion**.
 - **In packets** are dropped only if **congestion worsens** (queue exceeds Min_{in}).

Quality of Service

- Differentiated Services
 - The Assured Forwarding (AF) PHB



RED with In and Out drop probabilities

Summary

- We have discussed different resource allocation mechanisms
- We have discussed different queuing mechanisms
- We have discussed TCP Congestion Control mechanisms
- We have discussed Congestion Avoidance mechanisms
- We have discussed Quality of Services